

UNIVERSITÀ
degli STUDI
di CATANIA

DIPARTIMENTO DI INGEGNERIA
ELETTRICA ELETTRONICA E INFORMATICA

CORSO DI LAUREA IN INGEGNERIA INFORMATICA

Filippo Ficarra

IMPLEMENTAZIONE DI UN MOLTIPLICATORE A VIRGOLA FISSA APPROSSIMATO

Relatore:
Chiar.mo Prof. Giuseppe Ascia

Anno Accademico 2023/2024

INDICE

INTRODUZIONE 1

1. CONCETTI TEORICI SUL MOLTIPLICATORE A VIRGOLA FISSA 3

1.1. Moltiplicatore a virgola fissa esatto 3

1.2. Implementazione del moltiplicatore a virgola fissa tramite VHDL 7

2. STRUTTURA E IMPLEMENTAZIONE DEL MOLTIPLICATORE A VIRGOLA FISSA APPROSSIMATO 16

2.1. Confronto tra un moltiplicatore a virgola fissa esatto e approssimato .. 16

2.2. Tecniche di approssimazione per un moltiplicatore a virgola fissa approssimato 18

2.3. Modulo di correzione degli errori 22

2.4. Implementazione del moltiplicatore a virgola fissa approssimato 24

3. RISULTATI DEL MOLTIPLICATORE A VIRGOLA FISSA APPROSSIMATO 34

3.1. Prodotto di numeri generati casualmente 34

3.2. Prodotto di immagini 40

3.3. Calcolo dell'errore medio ed errore medio normalizzato 46

CONCLUSIONI 50

RINGRAZIAMENTI 51

INDICE DELLE FIGURE 52

INDICE DELLE TABELLE 55

BIBLIOGRAFIA 56

INTRODUZIONE

È noto come le operazioni aritmetiche svolgano un ruolo cruciale nell'elaborazione digitale dei segnali. Esse possono essere realizzate tramite due formati principali: il formato a virgola fissa e a virgola mobile.

Nonostante l'aritmetica a virgola mobile rappresenti un metodo vantaggioso per effettuare calcoli di numeri molto grandi o piccoli, l'aritmetica a virgola fissa risulta particolarmente utile nello svolgimento di calcoli approssimati, migliorando significativamente le prestazioni in termini di potenza, velocità ed efficienza energetica. Il frutto di tali vantaggi viene adoperato nel contesto di applicazioni tolleranti agli errori, come applicazioni multimediali, di riconoscimento immagini, apprendimento automatico e in ambito di audio, video e grafica.

Per realizzare un calcolo approssimato efficiente, si utilizzano moltiplicatori a virgola fissa approssimati. Sebbene un moltiplicatore a virgola fissa esatto garantisca precisione, introduce penalità significative in termini di area, tempo e consumo energetico, che possono essere mitigate utilizzando un moltiplicatore approssimato. Tale compromesso tra accuratezza ed efficienza è ideale per sistemi embedded e portatili con risorse limitate.

L'obiettivo di questa tesi è verificare i vantaggi offerti dall'utilizzo di un moltiplicatore a virgola fissa approssimato.

Nel primo capitolo si riportano cenni teorici sulla moltiplicazione a virgola fissa, evidenziando le differenze con quella a virgola mobile. Dopo aver definito il moltiplicatore a virgola fissa esatto, si descrive la sua implementazione tramite il linguaggio VHDL, per poi effettuarne la sintesi.

Il secondo capitolo descrive la struttura di un moltiplicatore a virgola fissa approssimato, dopo aver discusso dei possibili vantaggi rispetto all' impiego di un moltiplicatore esatto. Si illustrano le componenti utilizzate, ovvero gli adders approssimati, il compressore NAC e il modulo di correzione degli errori, per poi

procedere con la loro implementazione in VHDL, e realizzare, usufruendo dello stesso linguaggio, la struttura del moltiplicatore descritto. Si conclude, come nel capitolo precedente, effettuando la sintesi del componente descritto, evidenziando le principali differenze rispetto al moltiplicatore esatto.

Nel terzo capitolo, infine, si descrivono applicazioni pratiche del moltiplicatore a virgola fissa approssimato, effettuando il prodotto di numeri generati casualmente e di immagini, con lo scopo di evidenziare un errore medio e un errore medio normalizzato estremamente contenuti, oltre ad un'affinità visiva del prodotto di immagini, dal momento che le applicazioni grafiche dimostrano una maggiore tolleranza agli errori introdotti dall'approssimazione.

1. CONCETTI TEORICI SUL MOLTIPLICATORE A VIRGOLA FISSA

In questo primo capitolo, dopo aver introodotto la moltiplicazione a virgola fissa, viene definito il moltiplicatore esatto, per poi esaminarne l'implementazione software, usufruendo del linguaggio VHDL.

1.1. Moltiplicatore a virgola fissa esatto

L'elaborazione digitale dei segnali (DSP) viene effettuata grazie ad operazioni aritmetiche a virgola fissa e a virgola mobile, le quali determinano come i numeri vengono rappresentati e manipolati in un sistema digitale [1]. Nello specifico, in applicazioni DSP, come codec audio, filtri digitali, e altre applicazioni in tempo reale, in cui la velocità e l'efficienza energetica sono cruciali e la gamma dinamica richiesta non è particolarmente ampia, viene utilizzata una moltiplicazione a virgola fissa.
Essa prevede una rappresentazione dei numeri tramite un numero fisso di cifre per la parte intera e per la parte frazionaria, in maniera simile alla notazione decimale, ma su base binaria. La precisione è limitata dal numero di bit assegnati alla parte frazionaria. Non essendoci una gamma dinamica molto ampia, si incorre ad un rischio maggiore di overflow o underflow rispetto alla virgola mobile. Tuttavia, le operazioni aritmetiche a virgola fissa (come somma, sottrazione, moltiplicazione e divisione) godono di un' implementazione hardware favorita da una maggiore semplicità e velocità rispetto alle operazioni a virgola mobile, con una conseguente riduzione di consumo di risorse ed energia.

Un moltiplicatore a virgola fissa è un circuito digitale utilizzato per moltiplicare due numeri binari rappresentati in virgola fissa.

Esaminando il caso di un moltiplicatore ad 8 bit, si considerano come ingressi del circuito due vettori di 8 bit. Essi rappresentano il moltiplicatore ed il moltiplicando. L'uscita, invece, è un vettore di 16 bit, ovvero il prodotto dei due ingressi [1].

Questo tipo di moltiplicazione richiede vari passaggi. Il primo di tutti è la generazione dei prodotti parziali, dove ogni bit del moltiplicatore viene moltiplicato per ogni bit del moltiplicando. Questo genera una matrice di prodotti parziali, come si evince dalla figura 1.1. [1], [4].

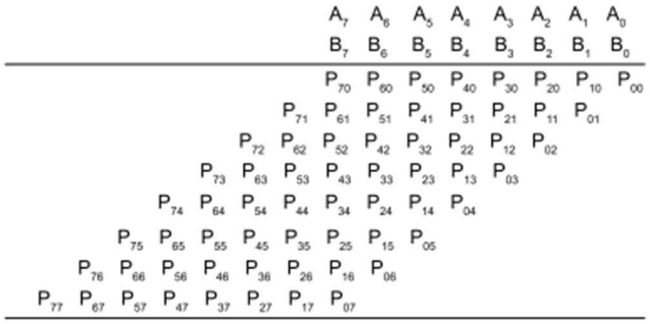


Figura 1.1. Matrice dei prodotti parziali [1], [4].

Successivamente, i bit del prodotto finale sono ottenuti sommando i bit del prodotto parziale. Tuttavia, le varie righe della matrice appena definita, sono prima compresse in due righe di bit, risultato ottenuto dopo vari stadi di compressione. Queste righe vengono sommate da un sommatore binario convenzionale, per poter generare i bit del prodotto finale. La compressione dei prodotti parziali richiede l'utilizzo di diverse componenti hardware:

- Half Adder
- Full Adder
- Compressori 4-2

Prima di definire di tali circuiti è opportuno sottolineare come, nelle formule utilizzate, gli operatori &, ^,~ e ⊕ corrispondono, rispettivamente, alle operazioni di AND, OR, NOT E XOR.

Un half adder è un circuito combinatorio che realizza la somma di due bit singoli, producendo due uscite: la somma *Sum* e il riporto *Carry*. Può essere implementato con un gate XOR (per la somma) e un gate AND (per il riporto).

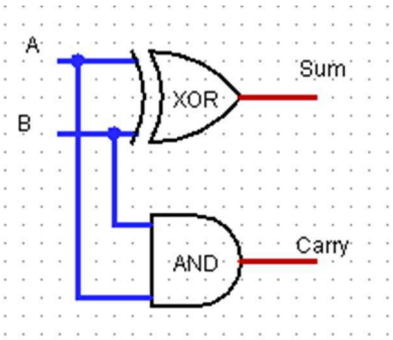


Figura 1.2 Circuito logico di un half adder.

Tabella 1.1. Tabella della verità relativa ad un half adder.

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Un full adder è un'estensione dell' half adder e permette di sommare tre bit: due bit di input A e B e un bit di riporto *carry in* proveniente da un'operazione precedente. Le uscite del full adder sono il bit somma *Sum* e il bit di riporto *Carry out* da passare all'operazione successiva.

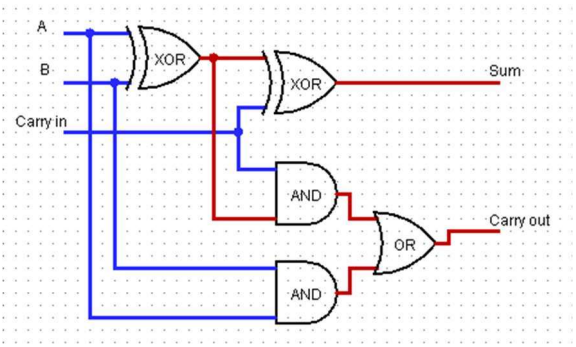


Figura 1.3. Circuito logico di un full adder

Tabella 1.2. Tabella della verità relativa ad un full adder.

A	B	Carry in	Sum	Carry out
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Un compressore 4-2 esatto, utilizzato per migliorare la velocità di compressione dei prodotti parziali, è realizzato tramite due full adder, come mostrato in figura 1.4 [5]. Gli ingressi sono cinque bit (A, B,C, D e *Carry in*) e le uscite sono tre bit, ovvero la somma *Sum* e i riporti *Cout* e *Carry* [6].

I cinque bit di input e il bit *Sum* hanno hanno tutti lo stesso peso di 2^0, mentre il secondo e il terzo bit di output *Carry* e *Cout* hanno il doppio di questo peso [1].

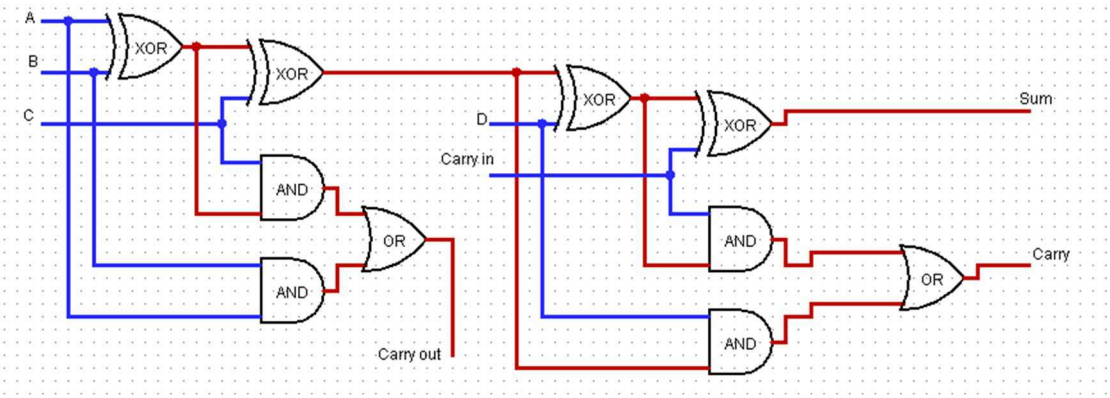


Figura 1.4. Circuito logico relativo ad un compressore 4-2.

I bit di uscita del compressore 4-2 sono calcolati secondo le seguenti equazioni:

$$\text{Sum} = \text{A} \oplus \text{B} \oplus \text{C} \oplus \text{D} \oplus \text{Carry in}$$

$$\text{Carry out} = C \;\&\; (A \oplus B) \;\hat{}\; A \;\&\; \sim (A \oplus B)$$

$$\text{Carry} = \text{Carry in} \;\&\; (A \oplus B \oplus C \oplus D) \;\hat{}\; \text{D} \;\&\; \sim (A \oplus B \oplus C \oplus D)$$

1.2. Implementazione del moltiplicatore a virgola fissa tramite VHDL

Si riporta, di seguito, l'implementazione software del moltiplicatore esatto ad 8 bit, ottenuta tramite il linguaggio VHDL (VHSIC Hardware Description Language), un linguaggio di descrizione hardware per modellare, simulare e sintetizzare circuiti digitali, descrivendo il comportamento di circuiti digitali a vari livelli di astrazione, dal livello di gate fino al livello architetturale [2].

La figura 1.5 analizza la strategia di implementazione del moltiplicatore esatto, in maniera simile a [4]. I rettangoli trattegiati con due e tre ingressi rappresentano, rispettivamente, gli half adder e i full adder, mentre i rettangoli non tratteggiati e con cinque ingressi costituiscono i compressori 4-2. Dopo aver generato la matrice di prodotti parziali, si effettuano i due stadi di compressione, ottenendo dei risultati intermedi, per poi assegnare al vettore d'uscita la somma dei risultati ottenuti dal secondo stadio di compressione [7].

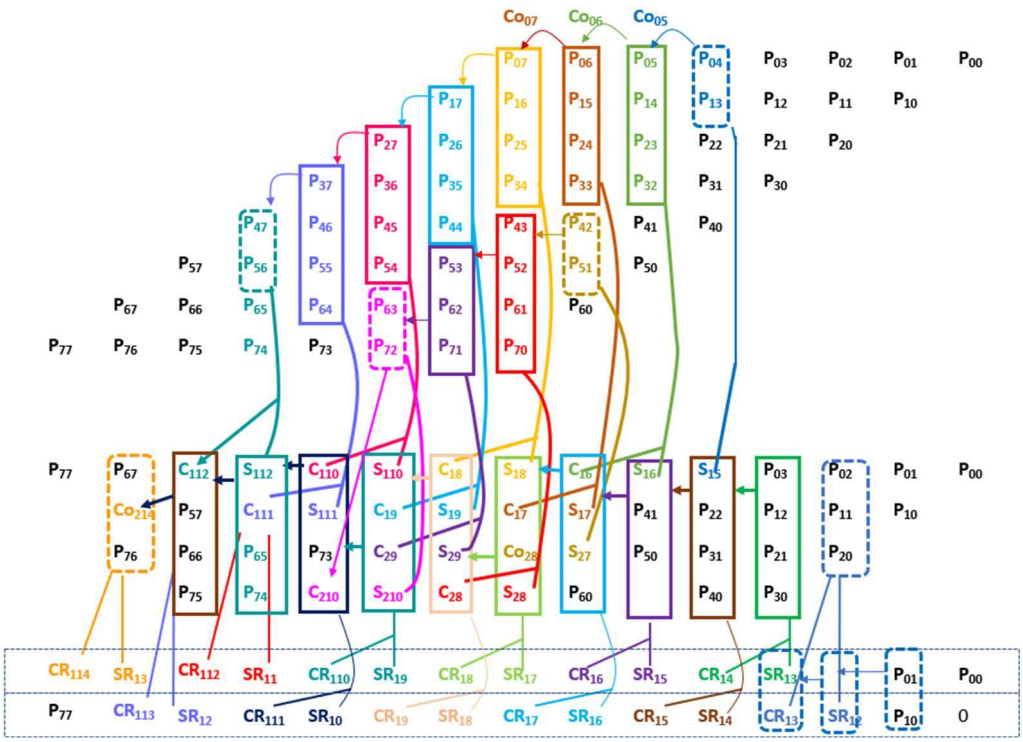


Figura 1.5. Logica dell'implementazione del moltiplicatore esatto.

Le seguenti figure riportano l'implementazione delle componenti utilizzate per realizzare un moltiplicatore fisso ad 8 bit, nonchè l'*half adder*, il *full adder* e il compressore 4-2.

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Half_Adder is
    Port (
        A : in STD_LOGIC;
        B : in STD_LOGIC;
        Sum : out STD_LOGIC;
        Cout : out STD_LOGIC
    );
end Half_Adder;

architecture Behavioral of Half_Adder is
begin
    Sum <= A xor B;
    Cout <= A and B;
end Behavioral;
```

Figura 1.6. Implementazione VHDL dell'half adder.

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Full_Adder is
    Port (
        A : in STD_LOGIC;
        B : in STD_LOGIC;
        Cin : in STD_LOGIC;
        Sum : out STD_LOGIC;
        Cout : out STD_LOGIC
    );
end Full_Adder;

architecture Behavioral of Full_Adder is
begin
    Sum <= A xor B xor Cin;
    Cout <= (A and B) or (Cin and (A xor B));
end Behavioral;
```

Figura 1.7. Implementazione VHDL del full adder

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Compressor4_2 is
    Port (
        A : in STD_LOGIC;
        B : in STD_LOGIC;
        C : in STD_LOGIC;
        D : in STD_LOGIC;
        Cin : in STD_LOGIC;
        Sum : out STD_LOGIC;
        Cout : out STD_LOGIC;
        Carry : out STD_LOGIC
    );
end Compressor4_2;

architecture Behavioral of Compressor4_2 is
begin
    Sum <= A xor B xor C xor D xor Cin;
    Cout <= (C and (A xor B)) or (A and (not(A xor B)));
    Carry <= (Cin and(A xor B xor C xor D))or(D and (not(A xor B xor C xor D)));
end Behavioral;
```

Figura 1.8. Implementazione VHDL del compressore 4-2.

Ogni implementazione presenta un' *entity* (interfaccia), la quale definisce l'interfaccia del componente, cioè gli ingressi e le uscite, e un' *architecture* (Architettura), che definisce il comportamento interno del modulo, ovvero come viene realizzata la funzione descritta nell'entity, mediante espressioni logiche, assegnazioni sequenziali, o istanze di altri moduli.

In particolar modo, gli ingressi e le uscite sono rappresentati utilizzando il tipo *std_logic*, allo scopo di raffigurare un singolo bit in un circuito digitale. Costituisce un vantaggio in termini di versatilità, rispetto al tipo *bit*, dal momento che può assumere più valori, mentre il secondo tipo citato può assumere solo i valori '0' ed '1'.

Si esamina l'implementazione del moltiplicatore ad 8 bit.

Dopo aver definito le librerie utilizzate, si definisce l'interfaccia (nominata "Multiplier_8bit") del moltiplicatore. Ha due ingressi (A e B), entrambi vettori di tipo *std_logic* di 8 bit, e un'uscita "Product" di 16 bit, che rappresenta il risultato della moltiplicazione.

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use ieee.std_logic_unsigned.all;

entity Multiplier_8bit is
    Port (
        A : in STD_LOGIC_VECTOR (7 downto 0);
        B : in STD_LOGIC_VECTOR (7 downto 0);
        Product : out STD_LOGIC_VECTOR (15 downto 0)
    );
end Multiplier_8bit;
```

Figura 1.9. Definizione delle librerie e dell'entity del moltiplicatore esatto ad 8 bit.

L'Architecture "Behavioral" definisce il comportamento interno del moltiplicatore, ma prima di definirlo, occorre specificare le componenti utilizzate, la cui implementazione è descritta nelle figure 1.6, 1.7 e 1.8.

```vhdl
architecture Behavioral of Multiplier_8bit is

    component Half_Adder
        Port (
            A : in STD_LOGIC;
            B : in STD_LOGIC;
            Sum : out STD_LOGIC;
            Cout : out STD_LOGIC
        );
    end component;

    component Full_Adder
        Port (
            A : in STD_LOGIC;
            B : in STD_LOGIC;
            Cin : in STD_LOGIC;
            Sum : out STD_LOGIC;
            Cout : out STD_LOGIC
        );
    end component;

    component Compressor4_2
        Port (
            A : in STD_LOGIC;
            B : in STD_LOGIC;
            C : in STD_LOGIC;
            D : in STD_LOGIC;
            Cin : in STD_LOGIC;
            Sum : out STD_LOGIC;
            Cout : out STD_LOGIC;
            Carry : out STD_LOGIC
        );
    end component;
```

Figura 1.10. Definizione delle componenti utilizzate.

```
48      type matrice is array (0 to 7, 0 to 7) of std_logic;
49      signal P: matrice;
50
51      type matrice2 is array (0 to 2, 0 to 15) of std_logic;
52      signal S, C, Co: matrice2;
53
54      signal SR1,CR1: std logic vector(15 downto 0);
```

Figura 1.11. Dichiarazioni di tipi e segnali.

I tipi e i segnali dichiarati mostrati in figura 1.11 sono:

- "matrice": un array bidimensionale di *std_logic*, usato per memorizzare i prodotti parziali.
- "matrice2": un array bidimensionale usato per memorizzare i risultati intermedi dei compressori.
- "P", "S", "C", "Co": segnali che rappresentano rispettivamente i prodotti parziali, le somme intermedie e i carry generati dai compressori.
- "SR1" e "CR1": vettori di 16 bit che rappresentano i risultati finali dei segnali intermedi, che vengono poi sommati per ottenere il prodotto finale.

```
56  begin
57
58      process(A,B)
59       begin
60          for i in 0 to 7 loop
61              for j in 0 to 7 loop
62                  P(i,j) <= A(j) and B(i);
63              end loop;
64          end loop;
65       end process;
```

Figura 1.12. Generazione dei prodotti parziali.

Come si evince dalla figura 1.12, la generazione dei prodotti parziali avviene tramite l'utilizzo dei *process*, nei quali, a differenza dell'assegnazione concorrente, le istruzioni vengono eseguite una dopo l'altra, nell'ordine in cui sono scritte. I *process* sono tipicamente associati a una lista di sensibilità (sensitivity list), che

elenca i segnali ai quali il process è sensibile. Il *process* viene eseguito ogni volta che uno dei segnali nella lista cambia valore.

Il *process* calcola i prodotti parziali usando l'AND bit a bit tra ciascun bit di A e B. Il risultato è memorizzato nella matrice P, la quale chiaramente avrà 64 elementi [10].

Volendo analizzare come vengono effettuati i vari stadi di compressione, per generare i risultati intermedi, è opportuno definire il costrutto *port map.* Esso è utilizzato per collegare i segnali esterni di un'entity ai segnali interni di un'altra entity o componente. È il meccanismo che permette di "mappare" i segnali o le porte di un'istanza di un componente ai segnali presenti nel contesto in cui quel componente viene utilizzato. Nella figura 1.13, per esempio, viene utilizzato l'Half Adder, che somma P (0,4) e P (1,3), per ottenere la somma S (1,5) e il riporto Co (0,5).

Le righe di codice riportate nelle figure 1.13 e 1.14 consentono di generare i risultati intermedi, effettuando i vari stadi di compressione tramite il costrutto *port map.*

I vettori SR1 e CR1 contengono i risultati intermedi finali, che vengono sommati per ottenere il prodotto finale, assegnato all'uscita "Product" [11].

```
68  --Primo stadio di compressione
69  HA: Half_Adder port map (P(0,4), P(1,3), S(1,5), Co(0,5));
70  C1: Compressor4_2 port map(P(0,5),P(1,4), P(2,3), P(3,2), Co(0,5), S(1,6), Co(0,6), C(1,6));
71  C2: Compressor4_2 port map(P(0,6),P(1,5), P(2,4), P(3,3), Co(0,6), S(1,7), Co(0,7), C(1,7));
72  C3: Compressor4_2 port map(P(0,7),P(1,6), P(2,5), P(3,4), Co(0,7), S(1,8), Co(0,8), C(1,8));
73  C4: Compressor4_2 port map(P(1,7),P(2,6), P(3,5), P(4,4), Co(0,8), S(1,9), Co(0,9), C(1,9));
74  C5: Compressor4_2 port map(P(2,7),P(3,6), P(4,5), P(5,4), Co(0,9), S(1,10), Co(0,10), C(1,10));
75  C6: Compressor4_2 port map(P(3,7),P(4,6), P(5,5), P(6,4), Co(0,10), S(1,11), Co(0,11), C(1,11));
76  FA: Full_Adder port map(P(4,7), P(5,6), Co(0,11), S(1,12), C(1,12));
77  HA2: Half_Adder port map(P(4,2), P(5,1), S(2,7), Co(1,7));
78  C7: Compressor4_2 port map(P(4,3),P(5,2), P(6,1), P(7,0), Co(1,7), S(2,8), Co(1,8),C(2,8));
79  C7_3: Compressor4_2 port map(P(5,3),P(6,2), P(7,1), '0', Co(1,8), S(2,9), Co(1,9),C(2,9));
80  FA2: Full_Adder port map(P(6,3), P(7,2), Co(1,9), S(2,10), C(2,10));
```

Figura 1.13. Primo stadio di compressione dei prodotti parziali per un moltiplicatore esatto.

```
--Secondo stadio di compressione
FA4: Full_Adder port map(P(0,2), P(1,1), P(2,0), SR1(2), CR1(3));
C8: Compressor4_2 port map(P(0,3), P(1,2), P(2,1), P(3,0), '0', SR1(3), Co(2,5), CR1(4));
C9: Compressor4_2 port map(S(1,5),P(2,2), P(3,1), P(4,0),Co(2,5), SR1(4), Co(2,6), CR1(5));
C10: Compressor4_2 port  map(S(1,6),P(4,1),P(5,0), '0',Co(2,6), SR1(5),Co(2,7), CR1(6));
C11: Compressor4_2 port map(C(1,6),S(1,7),S(2,7),P(6,0),Co(2,7),SR1(6),Co(2,8), CR1(7 ));
C12: Compressor4_2 port map(S(1,8),C(1,7),'0',S(2,8),Co(2,8), SR1(7), Co(2,9), CR1(8));
C13: Compressor4_2 port map(C(1,8),S(1,9),S(2,9),C(2,8), Co(2,9), SR1(8),Co(2,10), CR1(9));
C14: Compressor4_2 port map(S(1,10),C(1,9),C(2,9),S(2,10), Co(2,10), SR1(9), Co(2,11),CR1(10));
C15: Compressor4_2 port map(C(1,10), S(1,11), P(7,3), C(2,10), Co(2,11), SR1(10), Co(2,12), CR1(11));
C16: Compressor4_2 port map(S(1,12),C(1,11), P(6,5), P(7,4), Co(2,12), SR1(11), Co(2,13), CR1(12));
C17: Compressor4_2 port map(C(1,12),P(5,7),P(6,6), P(7,5), Co(2,13), SR1(12), Co(2,14), CR1(13));
FA5: Full_Adder port map(P(6,7), Co(2,14), P(7,6), SR1(13),CR1(14));


            SR1(0) <= P(0,0);
            SR1(1) <= P(1,0);
            SR1(14) <= P(7,7);
            SR1(15) <= '0';

            CR1(0) <= '0';
            CR1(1) <= P(0,1);
            CR1(2) <= '0';
            CR1(15) <= '0';

            Product <= SR1+ CR1;


end Behavioral;
```

Figura 1.14. Secondo stadio di compressione e assegnazione del vettore "Product" per un moltiplicatore esatto.

La figura 1.15 mostra la simulazione del moltiplicatore esatto a 8 bit, eseguita utilizzando il software Vivado [3].

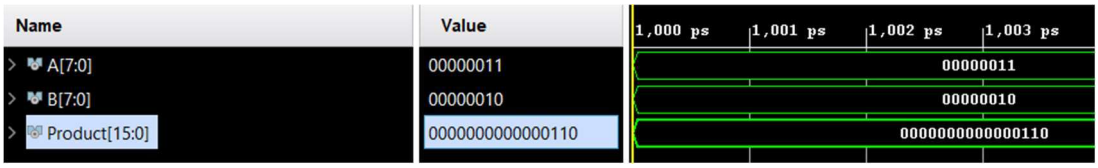


Figura 1.15. Simulazione di un moltiplicatore esatto a 8 bit.

Si esegue adesso la sintesi del moltiplicatore appena descritto, realizzata tramite Vivado [3], analizzando i seguenti risultati:

- Layout fisico della FPGA.

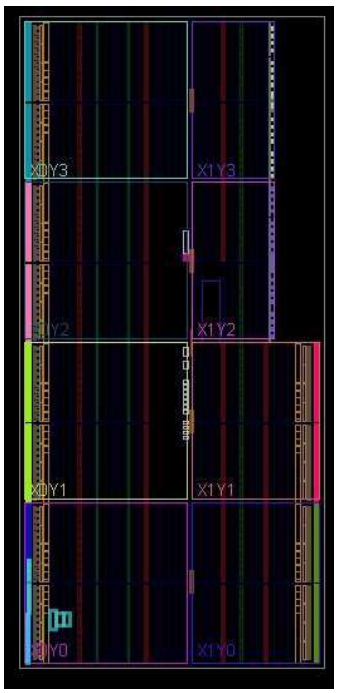


Figura 1.16. Layout del dispositivo del moltiplicatore esatto.

- consumo di potenza del design

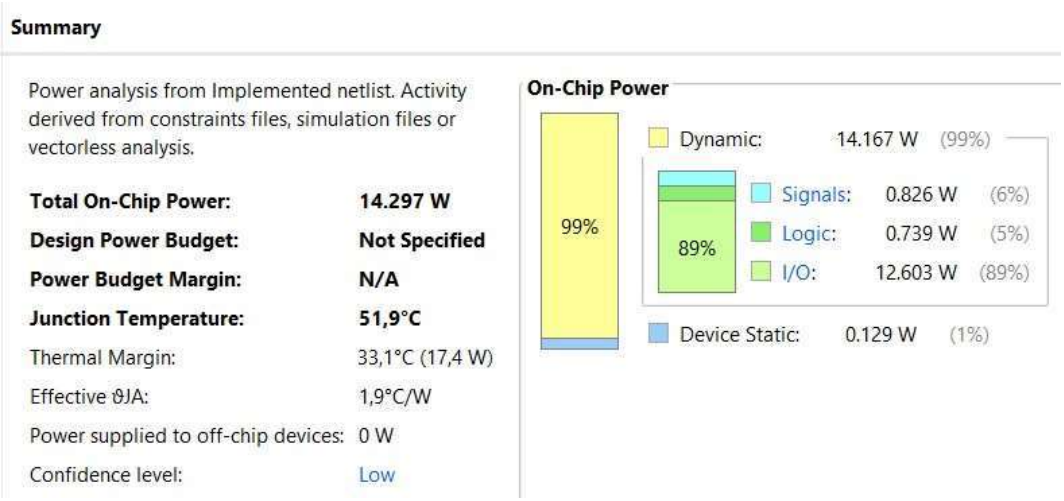


Figura 1.17. Analisi della potenza del moltiplicatore esatto.

- Schema logico del circuito logico sintetizzato, nonché una rappresentazione a livello di gate del moltiplicatore, utile per comprendere la struttura logica e i percorsi del segnale.

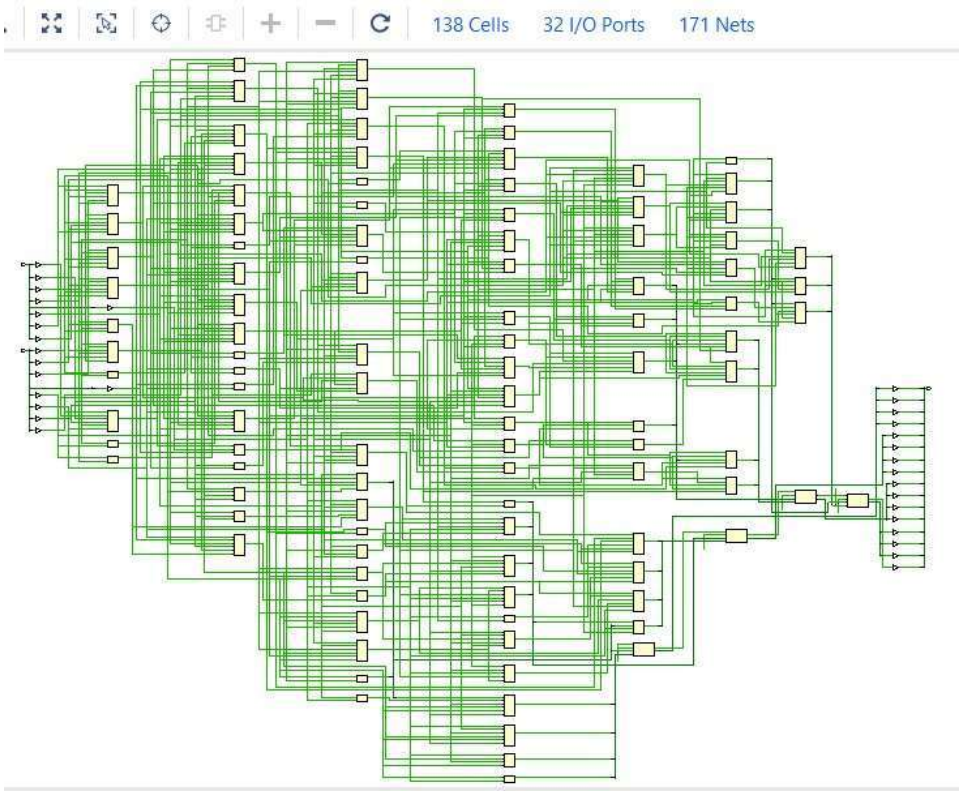


Figura 1.18. Schema logico del moltiplicatore esatto.

- Allocazione delle risorse logiche utilizzate dal moltiplicatore:
    1. 81 LUT (Look-Up Table), usate per implementare la logica del moltiplicatore. Le LUTs sono blocchi fondamentali di logica programmabile in una FPGA.
    2. 23 Slice, cioè gruppi di risorse logiche.
    3. 32 IOB (Input/Output Blocks), utilizzati per gestire i segnali in ingresso e in uscita, necessari per la comunicazione del moltiplicatore con altri blocchi.

Name 1	Slice LUTs (41000)	Slice (10250)	LUT as Logic (41000)	Bonded IOB (300)
N Multiplier_8bit	81	23	81	32

Figura 1.19. Utilizzo delle risorse del moltiplicatore esatto.

2. STRUTTURA E IMPLEMENTAZIONE DEL MOLTIPLICATORE A VIRGOLA FISSA APPROSSIMATO

Il seguente capitolo intende riportare nel dettaglio la struttura di un moltiplicatore a virgola fissa approssimato, esaminando le componenti utilizzate per assemblarlo e la sua implementazione software, usufruendo, come in precedenza, del linguaggio VHDL.

2.1. Confronto tra un moltiplicatore a virgola fissa esatto e approssimato

Il moltiplicatore a 8 bit, analizzato nel paragrafo precedente, effettua un calcolo accurato, dato dalla rappresentazione a virgola fissa. Ogni bit del risultato è determinato con precisione, garantendo il mantenimento dell' informazione, senza correre il rischio di eventuali perdite.

Se la precisione rappresenta il vantaggio dell'utilizzo del moltiplicatore esatto, è importante sottolineare come tale componente introduca penalità significative in termini di tempo, area e consumo energetico.

Si tratta, infatti, di un circuito generalmente più lento rispetto a un moltiplicatore approssimato, poiché esegue tutte le operazioni necessarie per garantire un risultato preciso. Basti pensare alla possible richiesta di cicli di clock aggiuntivi o pipeline più lunghe, a seconda della complessità dell'operazione e della lunghezza dei numeri coinvolti. Allo stesso tempo, richiede più risorse hardware, come LUTs (Look-Up Tables), registri e blocchi DSP (Digital Signal Processing) in un FPGA, o più transistor in un ASIC, per implementare l'operazione di moltiplicazione precisa. L'area necessaria aumenta con la lunghezza dei numeri da moltiplicare, soprattutto se sono necessarie tecniche di gestione dell'overflow o

dell'arrotondamento. In aggiunta a ciò, consuma più energia poiché richiede più operazioni logiche e di memorizzazione. Il consumo energetico cresce proporzionalmente alla complessità del calcolo e alla precisione richiesta.

Proprio per i motivi appena elencati, si decide di offrire un compromesso accettabile tra accuratezza ed efficienza, utilizzando un moltiplicatore a virgola fissa approssimato, introducendo delle semplificazioni nell'algoritmo di moltiplicazione, con l'obiettivo di ridurre la complessità e la precisione. La perdita di precisione dipende dall'approccio utilizzato per l'approssimazione.

Volendo confrontare le prestazioni in termini di tempo, area e consumo energetico, il moltiplicatore approssimato offre generalmente una maggiore velocità di esecuzione, dal momento che, eliminando alcuni bit o utilizzando tecniche di calcolo semplificato, riduce la complessità dei calcoli interni. Inoltre, occupa meno area hardware rispetto a un moltiplicatore esatto, grazie alle semplificazioni introdotte, rendendo possibile l'implementazione su hardware più piccoli ed economici.
Infine, utilizzando meno risorse hardware ed eseguendo meno operazioni, consuma meno energia rispetto al moltiplicatore esatto, motivo per cui spesso viene utilizzato in dispositivi IoT o in sistemi portatili.

L'utilizzo di un moltiplicatore esatto, dunque, è necessario in applicazioni che richiedono calcoli precisi e affidabili, come nel controllo automatico, nelle telecomunicazioni (modulazione e demodulazione), nei calcoli finanziari, e in molte altre applicazioni ingegneristiche critiche, proprio perchè errori di calcolo possono portare a malfunzionamenti o risultati inaccettabili.

Nelle applicazioni dove un certo livello di errore è accettabile o può essere compensato, come nell'elaborazione di audio o video o nel riconoscimento di immagini e nel machine learning (reti neurali approssimate), risulta, invece, conveniente utilizzare un moltiplicatore approssimato, in quanto offre un compromesso tra precisione, tempo di calcolo, area hardware e consumo energetico [5], [8], [9]. Esso fornisce, dunque, risultati sufficienti in applicazioni tolleranti agli

errori. Basti pensare ad una generica applicazione grafica, dove la percezione limitata della visione umana, consente l'utilizzo di moltiplicatore approssimato [1].

2.2. Tecniche di approssimazione per un moltiplicatore a virgola fissa approssimato

Alcune delle componenti utilizzate per realizzare un moltiplicatore a virgola fissa approssimato sono già note: si tratta degli adders esatti e del compressore 4-2 esatto. Si introducono adesso le nuove componenti approssimate.

Prima di procedere è opportuno sottolineare come il loro utilizzo introduca errori nel sistema, rappresentati dalla differenza tra l'output del componente approssimato e l'output del componente esatto [1].

L'half adder proposto e utilizzato nella costruzione del moltiplicatore presenta una singola porta OR a due ingressi, grazie alla quale si ottiene la somma approssimata dei due bit [1].

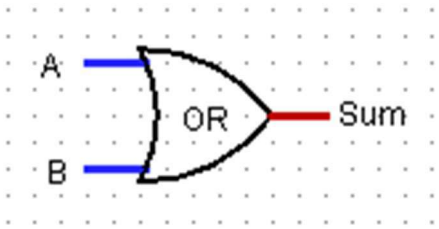


Figura 2.1. Circuito logico di un half adder approssimato.

Si consideri adesso il valore decimale della somma ottenuta dall'half adder esatto, rappresentato dalla formula

$$S = sum + 2 \cdot carry$$

Si denota, invece, con S', l'output ottenuto dall'half adder approssimato. Allora, in questo caso, il componente proposto genera un errore di -1 solo nel caso di somma di due bit di input diversi da 0, come mostrato in tabella 2.1

Tabella 2.1. Tabella della verità di un half adder approssimato e confronto con l'half adder esatto in termini di errore.

A	B	S'	S	Errore(S'-S)
0	0	0	0	0
0	1	1	1	0
1	0	1	1	0
1	1	1	2	-1

Il full adder approssimato, utilizzato per realizzare il moltiplicatore, è composto da una porta AND, una porta OR, e un *inverter* (una porta NOT) [1].

Il circuito legge tre bit di input e restituisce come output i due bit *Sum* e *Carry out,* dati dalle seguenti espressioni logiche:

$$\text{Sum} = \sim(\text{A}\hat{}(\text{B \& C}))$$

$$\text{Carry out} = \text{A}\hat{}(\text{B \& C})$$

Si sottolinea come il bit rappresentante il carry out abbia un peso doppio rispetto alla somma.

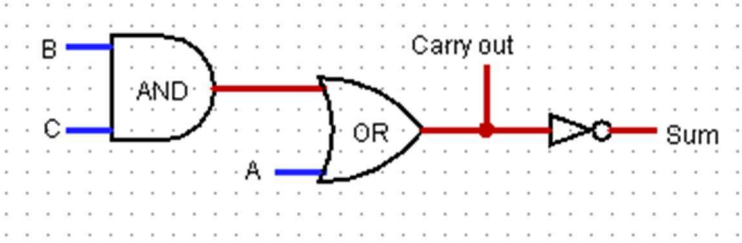


Figura 2.2. Circuito logico di un full adder approssimato.

In questo modo si riduce notevolmente l'area occupata, dal momento che il full adder esatto richiede, invece, una porta OR, due porte XOR e due porte AND. Inoltre, considerando le otto possibili combinazioni di bit di input, viene generato un errore solo in tre casi.

Tabella 2.2. Tabella della verità di un full adder approssimato e confronto con il full adder esatto in termini di errore.

A	B	C	Exact sum	Exact carry out	Approximate sum	Approximate carry out	Error
0	0	0	0	0	1	0	1
0	0	1	1	0	0	1	1
0	1	0	1	0	1	0	0
0	1	1	0	1	0	1	0
1	0	0	1	0	1	0	0
1	0	1	0	1	0	1	0
1	1	0	0	1	0	1	0
1	1	1	1	1	0	1	-1

Il compressore approssimato proposto, denominato con l'appellativo "NAC", riceve in ingresso 4 bit della matrice dei prodtti parziali, per restituire una somma in uscita, tramite l'utilizzo di tre porte OR, come si evince dalla figura 2.3 [1].

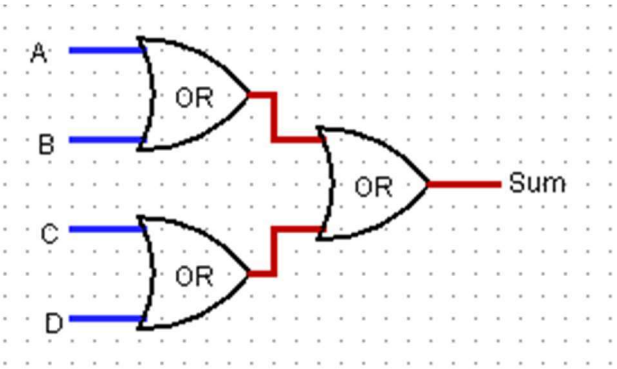


Figura 2.3. Circuito logico di un NAC.

La somma *sum* è quindi calcolata tramite la formula:

$$sum = (A \hat{} B) \hat{} (C \hat{} D)$$

Il design del NAC utilizza meno operazioni rispetto a un compressore esatto, portando a una minore area e consumo di potenza. Basti pensare alla differenza quantitativa con i cinque input e i due output del compressore esatto, il quale utilizza

due full adder esatti, nonchè quattro porte XOR, quattro porte AND e due porte OR, a differenza delle sole tre porte OR utilizzate nel compressore NAC.

Sebbene il NAC possa produrre risultati con errori rispetto a un compressore esatto, il suo utilizzo è giustificato in applicazioni tolleranti agli errori, come l'elaborazione di segnali o immagini, dove la precisione assoluta non è sempre necessaria.

Si veda a tal proposito la tabella 2.3, all'interno della quale vengono riportate le 2^4 combinazioni dei bit di ingresso, oltre alla loro somma, calcolata usando un compressore 4-2 esatto. Si mostrano anche la somma ottenuta dal compressore NAC e l'errore generato, calcolato come differenza tra l'uscita del compressore approssimato e l'uscita esatta:

$$NAC\ Error = NAC\ sum - \ Exact\ sum$$

Tabella 2.3. Tabella della verità del compressore NAC e confronto con il compressore 4-2 esatto in termini di errore.

A	B	C	D	Excact sum	NAC sum	NAC Error
0	0	0	0	0	0	0
0	0	0	1	1	1	0
0	0	1	0	1	1	0
0	0	1	1	2	1	-1
0	1	0	0	1	1	0
0	1	0	1	2	1	-1
0	1	1	0	2	1	-1
0	1	1	1	3	1	-2
1	0	0	0	1	1	0
1	0	0	1	2	1	-1
1	0	1	0	2	1	-1
1	0	1	1	3	1	-2
1	1	0	0	2	1	-1
1	1	0	1	3	1	-2
1	1	1	0	3	1	-2
1	1	1	1	4	1	-3

2.3. Modulo di correzione degli errori

È emerso come il compressore approssimato produca un errore negativo, considerando che l'output dei compressori approssimati è sempre inferiore rispetto all'output esatto.

Per mitigare questo errrore, viene adoperato il modulo di correzione degli errori (ECM, *Error Correction Module*), una componente utilizzata nei circuiti digitali approssimati. Il suo scopo è migliorare l'accuratezza del risultato finale, compensando parzialmente l'errore causato dalla logica approssimata utilizzata in precedenza nel circuito.

Il modulo di correzione degli errori interviene per aggiustare il risultato parziale prodotto dai compressori approssimati. Questi compressori sono progettati per ridurre la complessità del circuito a scapito della precisione. L'ECM cerca di ridurre l'impatto degli errori introdotti, soprattutto nelle posizioni di bit più significative, dove l'errore avrebbe un impatto maggiore sul risultato finale. Il modulo ECM, generalmente, produce un bit di correzione che viene aggiunto alla somma finale, generato sulla base di una logica che analizza le combinazioni degli ingressi e degli output intermedi del compressore.

L'ECM è progettato per rilevare specifici pattern di errore nei bit di input più significativi e quindi applicare una correzione. L'ECM riceve come input i bit parziali provenienti dai compressori o adders approssimati, in particolare quelli associati ai bit più significativi (MSB) del risultato [1]. Il modulo utilizza una logica combinatoria per determinare se è necessario introdurre un bit di correzione. Ad esempio, potrebbe rilevare se una specifica combinazione di bit in ingresso è suscettibile alla produzione di un errore significativo e, di conseguenza, generare un bit di correzione. L'output è un bit di correzione che viene sommato al bit più significativo del compressore successivo, o applicato direttamente al risultato finale, per compensare l'errore.

Basti pensare ad un compressore 4-2 approssimato che, in alcune combinazioni di input, produce un errore negativo, cioè un'uscita che è inferiore a quella corretta

[12]. L'ECM può essere configurato per rilevare queste specifiche combinazioni e aggiungere un bit di correzione che corregge l'errore parzialmente o completamente. In un'architettura di moltiplicatore approssimato, in cui compressori approssimati sono usati per ridurre la matrice di prodotto parziale, l'ECM viene inserito nei bit più significativi della matrice ridotta per rilevare e correggere errori critici prima che il risultato venga sommato e presentato come output.

Il modulo di correzione degli errori (ECM) è, dunque, una tecnica cruciale in architetture di circuiti approssimati, soprattutto quando si cerca di bilanciare l'efficienza con la precisione. Viene utilizzato per correggere errori critici che potrebbero compromettere la qualità del risultato finale. Sebbene aggiunga complessità al circuito, l'ECM rappresenta una soluzione efficace per applicazioni dove è accettabile un certo grado di approssimazione, mantenendo una qualità del risultato accettabile.

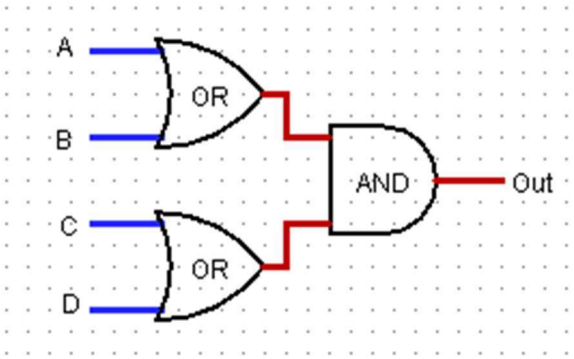


Figura 2.4. Circuito logico del modulo di correzione degli errori.

Si consideri la figura 2.4. I quattro ingressi sono i bit parziali derivati dalla catena di compressione dei prodotti parziali, nonchè i bit più significativi della somma parziale che si vuole correggere.

I risultati delle operazioni OR tra le due coppie di bit vengono poi combinati attraverso un'operazione logica AND, un passaggio cruciale, con lo scopo di rilevare specifici pattern che sono suscettibili e di generare errori significativi [5].

Il risultato dell'operazione AND finale è un singolo bit di correzione, che può essere 0 o 1, il quale viene poi utilizzato come ingresso (cin) per il compressore successivo o viene sommato alla parte più significativa del risultato parziale per correggere l'errore introdotto [1].

2.4. Implementazione del moltiplicatore a virgola fissa approssimato

Come nel caso precedente, si utilizza il linguaggio VHDL [2] per l'implementazione software del moltiplicatore a virgola fissa approssimato. Anche in questo caso gli ingressi sono rappresentati da due array di tipo *std_logic* di 8 bit, mentre l'uscita da un array *std_logic* di 16.

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_UNSIGNED.ALL;
4  use IEEE.NUMERIC_STD.ALL;
5
6
7  entity Approximate_Multiplier_8bit is
8    Port ( A : in STD_LOGIC_VECTOR (7 downto 0);
9           B : in STD_LOGIC_VECTOR (7 downto 0);
10          Product : out STD_LOGIC_VECTOR (15 downto 0) );
11 end Approximate_Multiplier_8bit;
12
```

Figura 2.5. Definizione delle librerie adoperate e dell'entity del moltiplicatore a virgola fissa approssimato.

Prima di realizzarne l'architettura interna, occorre definire le componenti discusse in precedenza: gli adders approssimati, il compressore NAC, e il modulo di correzione degli errori Sono illustrati, rispettivamente, nelle figure 2.6, 2.7, 2.8 e 2.9.

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;


entity Approximate_Half_Adder is
    Port(A,B: in std_logic;
    Sum: out std_logic);
end Approximate_Half_Adder;

architecture Behavioral of Approximate_Half_Adder is

begin
    Sum<=A or B;

end Behavioral;
```

Figura 2.6. Implementazione VHDL dell'half adder approssimato.

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Approximate_Full_Adder is
 Port (
    A,B,C: in std_logic;
    Sum, Cout: out std_logic);
end Approximate_Full_Adder;

architecture Behavioral of Approximate_Full_Adder is

begin
    Sum<=not(A or(B and C));
    Cout<=A or(B and C);
end Behavioral;
```

Figura 2.7. Implementazione VHDL del full adder approssimato.

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Approximate_Compressor_NAC is
  Port ( A,B,C,D: in std_logic;
         Sum: out std_logic);
end Approximate_Compressor_NAC;

architecture Behavioral of Approximate_Compressor_NAC is

begin
    Sum<=(A or B) or (C or D);

end Behavioral;
```

Figura 2.8. Implementazione VHDL del compressore NAC.

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Error_correction_module is
 Port (A,B,C,D: in std_logic;
       O: out std_logic );
end Error_correction_module;

architecture Behavioral of Error_correction_module is

begin
    O<= (A or B) and (C or D);

end Behavioral;
```

Figura 2.9. Implementazione VHDL del modulo di correzione degli errori.

Chiaramente le componenti sono implementate separatamente in file VHDL, distinti, pur rimanendo all'interno della medesima directory del progetto, conferendo così una struttura modulare ed elegante all'architettura complessiva. Tali circuiti sono, poi, definiti come componenti del moltiplicatore a virgola fissa approssimato, ancor prima di definire l'architettura di quest'ultimo, tramite il costrutto *component*.

Sono definiti anche gli adders e il compressore 4-2 esatto discussi in precedenza.

```
46      component Approximate_Half_Adder
47          Port(
48              A,B: in std_logic;
49              Sum: out std_logic
50          );
51      end component;
52
53      component Approximate_Full_Adder
54          Port (
55              A,B,C: in std_logic;
56              Sum, Cout: out std_logic
57          );
58
59      end component;
60
61      component Approximate_Compressor_NAC
62              Port ( A,B,C,D: in std_logic;
63                      Sum: out std_logic
64              );
65      end component;
66
67      component Error_correction_module
68               Port (A,B,C,D: in std_logic;
69                      O: out std_logic
70                );
71      end component;
```

Figura 2.10. Dichiarazione dell'uso di componenti approssimate e del modulo di correzione degli errori.

A questo punto, la dichiarazione dei segnali adoperati, come la matrice di segnali P (memorizza i prodotti parziali della moltiplicazione) le matrici S, C, e Co (conservano i risultati intermedi e i carry-out generati durante la compressione dei bit) e i vettori SR1 e CR1 (immagazzinano rispettivamente i risultati e i carry-out finali) condividono l'implementazione adoperata nel moltiplicatore esatto (vedasi figura 1.11), così come anche la generazione dei prodotti parziali (vedasi figura 1.12).

Ciò che distingue un moltiplicatore a virgola fissa approssimato da uno esatto è la compressione dei bit del prodotto parziale.

Si analizzi la seguente figura, la quale riporta uno schema di compressione simile a [5]. I rettangoli tratteggiati rappresentano le componenti approssimate e i rettangoli non trattegiate le componenti esatte:

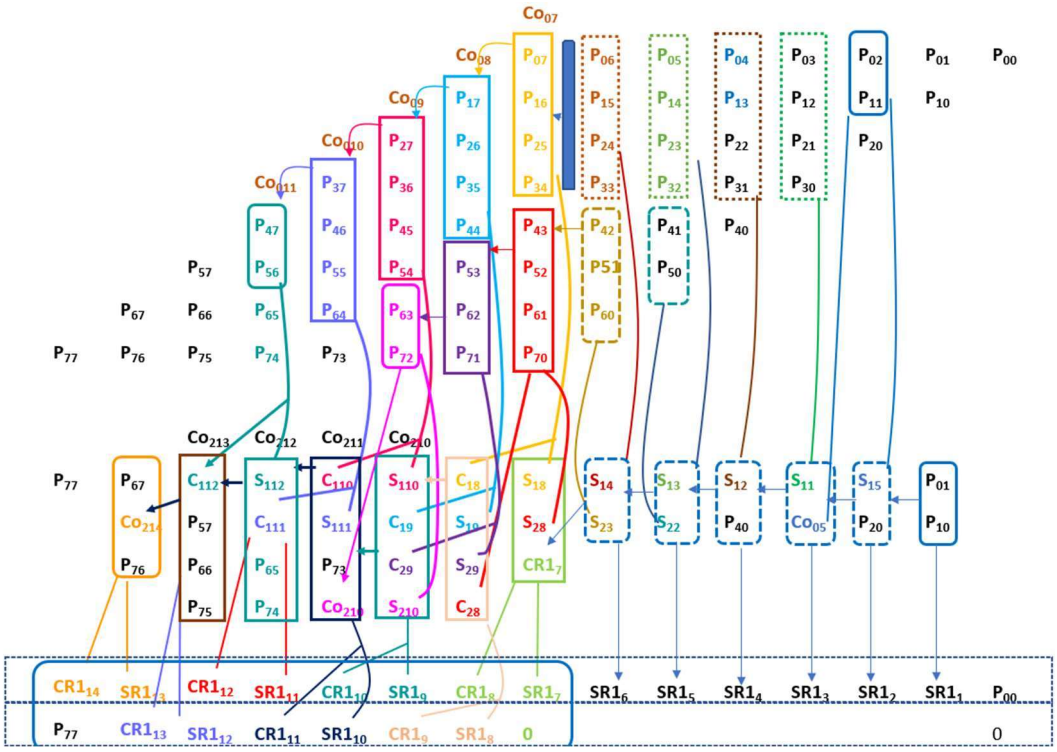


Figura 2.11. Logica della compressione della matrice di bit dei prodotti parziali del moltiplicatore approssimato.

Nella metà desta della matrice dei bit del prodotto parziale, i compressori esatti vengono sostituiti da sommatori (half adders e full adders) approssimati e compressori NAC.

Si nota che la metà sinistra della matrice di bit del prodotto parziale utilizza esclusivamente calcoli esatti per evitare di generare un grande errore nel prodotto di output. Pertanto, le componenti approssimate vengono utilizzate solo nella metà destra della matrice dei bit del prodotto parziale.

Il modulo ECM viene, invece, utilizzato per compensare l'errore introdotto. Si collega l'output del modulo ECM come input di carry al compressore esatto successivo a sinistra. Il modulo ECM rileva il pattern di input del MSB della catena del compressore utilizzando due porte OR seguite da una porta AND e genera un bit di correzione di 0 o 1 di conseguenza [1].

Si riporta di seguito l'implementazione su VHDL della compressione dei bit:

```
--Primo stadio di compressione

            --metà  destra della matrice di prodotti parziali

            HA: Half_Adder port map (P(0,2), P(1,1), S(1,5), Co(0,5));
            NAC: Approximate_Compressor_NAC port map(P(0,3),P(1,2),P(2,1),P(3,0),S(1,1));
            NAC2: Approximate_Compressor_NAC port map(P(0,4),P(1,3),P(2,2),P(3,1),S(1,2));
            NAC3: Approximate_Compressor_NAC port map(P(0,5),P(1,4),P(2,3),P(3,2),S(1,3));
            NAC4: Approximate_Compressor_NAC port map(P(0,6),P(1,5),P(2,4),P(3,3),S(1,4));
            ECM: Error_correction_module port map(P(0,6),P(1,5),P(2,4),P(3,3), Co(0,7));
            AHA: Approximate_Half_Adder port map(P(4,1),P(5,0), S(2,2));
            AFA: Approximate_Full_Adder port map(P(4,2),P(5,1),P(6,0), S(2,3), Co(1,7));

            --metà sinistra della matrice di prodotti parziali
          C1: Compressor4_2 port map(P(0,7),P(1,6), P(2,5), P(3,4), Co(0,7), S(1,8), Co(0,8), C(1,8));
          C2: Compressor4_2 port map(P(1,7),P(2,6), P(3,5), P(4,4), Co(0,8), S(1,9), Co(0,9), C(1,9));
          C3: Compressor4_2 port map(P(2,7),P(3,6), P(4,5), P(5,4), Co(0,9), S(1,10), Co(0,10), C(1,10));
          C4: Compressor4_2 port map(P(3,7),P(4,6), P(5,5), P(6,4), Co(0,10), S(1,11), Co(0,11), C(1,11));
          FA: Full_Adder port map(P(4,7), P(5,6), Co(0,11), S(1,12), C(1,12));
          C5: Compressor4_2 port map(P(4,3),P(5,2), P(6,1), P(7,0), Co(1,7), S(2,8), Co(1,8),C(2,8));
          C6: Compressor4_2 port map(P(5,3),P(6,2), P(7,1), '0', Co(1,8), S(2,9), Co(1,9),C(2,9));
          FA2: Full_Adder port map(P(6,3), P(7,2), Co(1,9), S(2,10), C(2,10));
```

Figura 2.12. Primo stadio di compressione della matrice di prodotti parziali per il moltiplicatore approssimato.

```
--Secondo stadio di compressione

    --metà  destra
          HA2: Half_Adder port map(P(0,1),P(1,0),SR1(1),CR1(2));
          AFA2: Approximate_Full_Adder port map(S(1,5),P(2,0),CR1(2),SR1(2),CR1(3));
          AFA3: Approximate_Full_Adder port map(Co(0,5),S(1,1),CR1(3),SR1(3),CR1(4));
          AFA4: Approximate_Full_Adder port map(P(4,0),S(1,2),CR1(4),SR1(4),CR1(5));
          AFA5: Approximate_Full_Adder port map(S(2,2),S(1,3),CR1(5),SR1(5),CR1(6));
          AFA6: Approximate_Full_Adder port map(S(2,3),S(1,4),CR1(6),SR1(6),CR1(7));

     --metà  sinistra
          FA7: Full_Adder port map(S(1,8), S(2,8), CR1(7), SR1(7),CR1(8));
          C8: Compressor4_2 port map(C(1,8),S(1,9),S(2,9),C(2,8), '0', SR1(8),Co(2,10), CR1(9));
          C9: Compressor4_2 port map(S(1,10),C(1,9),C(2,9),S(2,10), Co(2,10), SR1(9), Co(2,11),CR1(10));
          C10: Compressor4_2 port map(C(1,10), S(1,11), P(7,3), C(2,10), Co(2,11), SR1(10), Co(2,12), CR1(11));
          C11: Compressor4_2 port map(S(1,12),C(1,11), P(6,5), P(7,4), Co(2,12), SR1(11), Co(2,13), CR1(12));
          C12: Compressor4_2 port map(C(1,12),P(5,7),P(6,6), P(7,5), Co(2,13), SR1(12), Co(2,14), CR1(13));
          FA3: Full_Adder port map(P(6,7), Co(2,14), P(7,6), SR1(13),CR1(14));
```

Figura 2.13. Secondo stadio di compressione della matrice di prodotti parziali per il moltiplicatore approssimato.

Si analizza ora la fase finale del moltiplicatore approssimato a 8 bit, in cui i risultati parziali calcolati nei passaggi precedenti vengono combinati per formare il prodotto finale.

```
136             SR1(0) <= P(0,0);
137             SR1(14) <= P(7,7);
138             SR1(15) <= '0';
139             CR1(15) <= '0';
140
141             Product(0)<=SR1(0);
142             Product(1)<=SR1(1);
143             Product(2)<=SR1(2);
144             Product(3)<=SR1(3);
145             Product(4)<=SR1(4);
146             Product(5)<=SR1(5);
147             Product(6)<=SR1(6);
148
149             Product(15 downto 7) <= SR1(15 downto 7) + CR1(15 downto 7);
150
151 end Behavioral;
```

Figura 2.14. Assegnazione finale dell'array "Product" per un moltiplicatore approssimato.

Il primo bit del vettore SR1 (che contiene il risultato parziale della moltiplicazione) viene assegnato a P (0,0). Questo rappresenta il bit meno significativo del prodotto, che è il risultato dell'operazione A (0) AND B (0) (il bit meno significativo di A moltiplicato per il bit meno significativo di B). Il quattordicesimo bit di SR1 (quasi il più significativo) viene assegnato a P (7,7), il prodotto del bit più significativo di A e B (ovvero A (7) AND B (7)). Questo bit è uno dei più alti nel risultato della moltiplicazione. Il bit più significativo del vettore SR1 (bit 15) viene impostato a 0. Questo indica che non ci sono bit di overflow significativi generati nei calcoli precedenti che necessitano di essere considerati per il bit più alto. Anche il quindicesimo bit del vettore CR1 (che contiene i carry-out) viene impostato a 0. Ciò indica che non ci sono carry-out provenienti dai calcoli precedenti per il bit più significativo.

In riferimento alla figura 2.14, si assegnano, dunque, ai primi 7 bit del prodotto finale, da Product (0) fino a Product (6), i corrispondenti valori del vettore SR1. Questi bit rappresentano i risultati diretti delle somme parziali già calcolate e non richiedono ulteriori operazioni. I bit più significativi del prodotto, ovvero da Product (7) a Product (15), sono ottenuti sommando i bit corrispondenti dei vettori

SR1(15 downto 7) (i risultati parziali) e CR1(15 downto 7) (i carry-out generati nelle compressioni precedenti). Questa somma finale include eventuali carry-out e rifinisce il risultato del prodotto approssimativo.

Una volta realizzata l'implementazione del moltiplicatore a virgola fissa approssimato, si riporta, utilizzando il software Vivado [3], la simulazione del prodotto di alcuni vettori, evidenziando le differenze di risultato tra i due moltiplicatori coinvolti:



Figura 2.15. Simula zione del prodotto di alcuni array a 8 bit tramite un moltiplicatore approssimato.

In figura 2.15, la discrepanza tra il risultato ottenuto mediante un moltiplicatore esatto e quello prodotto da un moltiplicatore approssimato risulta esigua, poiché la differenza relativa è attenuata dalla grandezza intrinseca dei valori risultanti. La natura elevata dei numeri generati dal prodotto tende, infatti, a ridurre l'incidenza degli errori introdotti dall'approssimazione, rendendoli trascurabili rispetto alla magnitudine complessiva del calcolo. Infatti, convertendo i vettori in formato decimale, si ha il prodotto tra i numeri 201 e 87, ottenendo un risultato esatto di 17487 e uno approssimato di 17551.

Si esegue, anche in questo caso, tramite Vivado [3], la sintesi del moltiplicatore appena descritto, analizzando i seguenti risultati:

- Allocazione delle risorse logiche utilizzate dal moltiplicatore, descritte nella sezione 1.2.

Name	Slice LUTs (41000)	Slice (10250)	LUT as Logic (41000)	Bonded IOB (300)
Approximate_Multiplier_8bit	66	19	66	32

Figura 2.16. Utilizzo delle risorse del moltiplicatore approssimato.

- Layout fisico della FPGA.

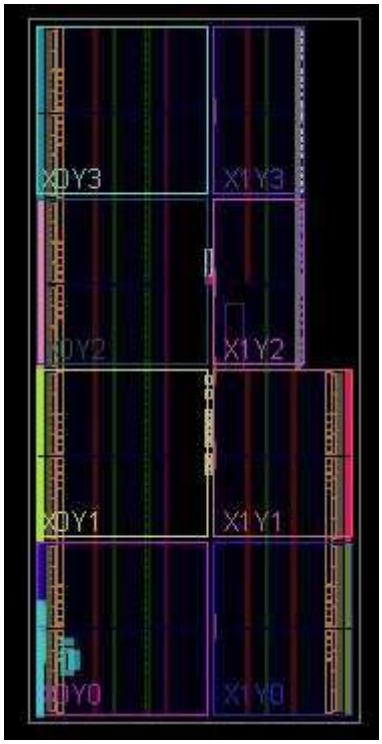


Figura 2.17. Layout del dispositivo del moltiplicatore aprprossimato.

- consumo di potenza del design

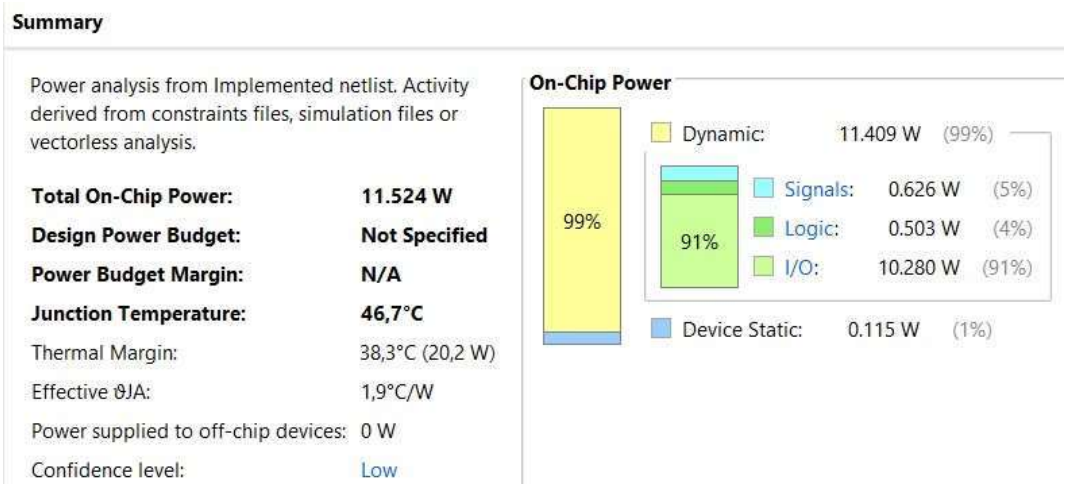


Figura 2.18. Analisi della potenza del moltiplicatore approssimato.

- Schema logico del circuito logico sintetizzato.

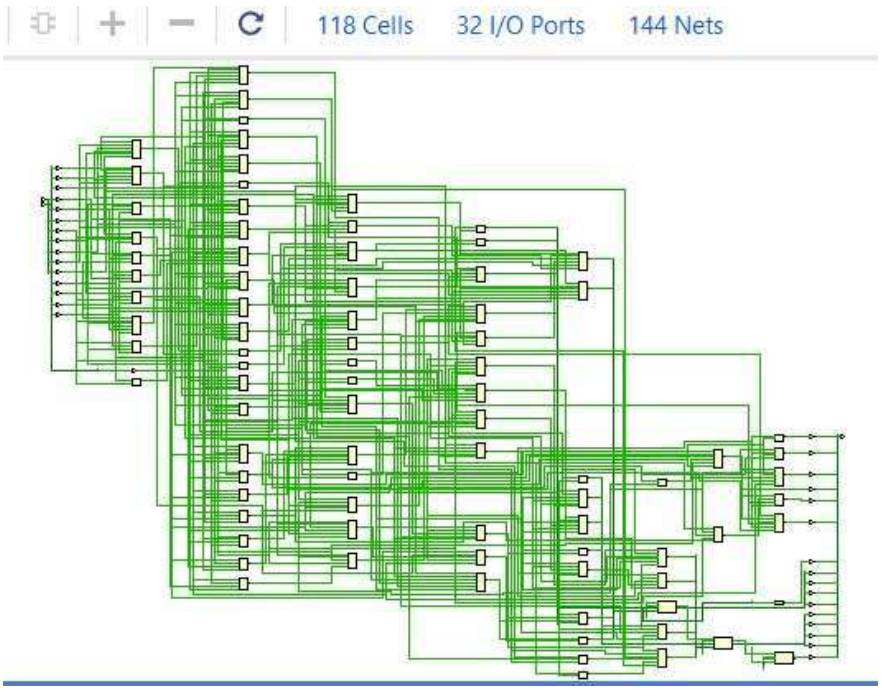


Figura 2.19. Schema logico del moltiplicatore approssimato.

Si evidenziano, dunque, le differenze tra i due diversi moltiplicatori in termini di consumo di potenza e utilizzo delle risorse, nonché area occupata.

Tabella 2.4. Confronto prestazionale tra il moltiplicatore esatto e approssimato

	Potenza statica	Potenza dinamica	Potenza totale	LUTs	Slice
Moltiplicatore esatto	0.129 W	14.167 W	14.297 W	81	23
Moltiplicatore approssimato	0.115 W	11.409 W	11.524 W	66	19

3. RISULTATI DEL MOLTIPLICATORE A VIRGOLA FISSA APPROSSIMATO

In questo capitolo conclusivo si analizzano alcune applicazioni del moltiplicatore a virgola fissa approssimato: il prodotto di numeri generati casualmente e di immagini. Si mette in luce come, con l'aumentare dell'ordine di rappresentazione numerica, l'errore medio e l'errore medio normalizzato risultino estremamente contenuti. Nel caso del prodotto di immagini, si evidenzia, inoltre, che i risultati ottenuti mediante moltiplicazione esatta e approssimata sono tra loro sorprendentemente affini, poiché le applicazioni grafiche dimostrano una maggiore tolleranza agli errori introdotti dall'approssimazione.

3.1. Prodotto di numeri generati casualmente

Un primo collaudo del moltiplicatore a virgola fissa approssimato riguarda il prodotto di alcuni numeri.

Un testbench, un ambiente di simulazione utilizzato per verificare il funzionamento e la correttezza di circuiti elettronici o di componenti hardware (in questo caso i due moltiplicatori) [2], genera numeri pseudo casuali, per i quali si esegue il prodotto con i due moltiplicatori descritti nei capitoli precedenti. I numeri generati e i due diversi prodotti vengono sovrascritti in un file di testo.

Nelle immagini riportate all’interno del paragrafo si analizza il codice relativo al testbench.

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.std_logic_arith.all;
4  use ieee.std_logic_unsigned.all;
5
6  use ieee.MATH_REAL.all;
7  use std.textio.all;
8  use ieee.std_logic_textio.all;
9
10 entity tb_integer is
11 end tb_integer;
12
13 architecture tbbrand_integer of tb_integer is
14
15     signal A, B: std_logic_vector(7 downto 0);
16     signal Ris_esatto, Ris_approssimato : std_logic_vector(15 downto 0) := (others => '0');
```

Figura 3.1. Definizione delle librerie e dell'entity del testbench per il prodotto di numeri generati casualmente.

Dopo aver definito le librerie necessarie per gestire tipi di dati logici e numerici, definire e gestire le operazioni aritmetiche, oltre la lettura e la scrittura per segnali *std_logic* e *std_logic_vector,* si definisce l'entità del testbench chiamata "tb_integer". Non ha porte perché, essendo un testbench, non ci sono interfacce esterne.

Si riporta, inoltre, l'architettura "tbbrand_integer", la quale definisce il comportamento del testbench, i segnali A e B, a 8 bit, che rappresentano i numeri da motliplicare, e i segnali "Ris_esatto" e "Ris_approssimato", a 16 bit, che immagazzinano rispettivamente il risultato esatto e quello approssimato della moltiplicazione.

```
19     component Multiplier_8bit
20         port (
21             A, B : in std_logic_vector(7 downto 0);
22             Product : out std_logic_vector(15 downto 0)
23         );
24     end component;
25
26     component Approximate_Multiplier_8bit
27         port (
28             A, B : in std_logic_vector(7 downto 0);
29             Product : out std_logic_vector(15 downto 0)
30         );
31     end component;
```

Figura 3.2. Definizione delle componenti utilizzate nel testbench.

In figura 3.2 si dichiarano le componenti che implementano la moltiplicazione. Uno fornisce il risultato esatto e l'altro un risultato approssimato.

```
33  begin
34
35      UUT1: Multiplier_8bit
36          port map (
37              A => A,
38              B => B,
39              Product => Ris_esatto
40          );
41
42
43      UUT2: Approximate_Multiplier_8bit
44          port map (
45              A => A,
46              B => B,
47              Product => Ris_approssimato
48          );
```

Figura 3.3. Instanza dei due moltiplicatori utilizzati nel testbench.

Si istanziano, successivamente, i due moltiplicatori. “UUT1” fornisce il risultato esatto, mentre “UUT2” quello approssimato. Entrambi prendono gli stessi input (A e B), ma producono diversi output (“Ris_esatto” e “Ris_approssimato”).

In VHDL, l'istanza di un componente è il processo con cui si collega una specifica unità di design (come un modulo o un blocco di moltiplicazione) al resto del circuito o sistema. L'istanza si ottiene mappando le porte del componente ai segnali del testbench, usando il costrutto *port map*, che collega i segnali interni del testbench alle porte del componente, come è possible vedere in figura 3.3.

La generazione dei numeri casuali, e il loro prodotto tramite i due moltiplicaotri, inizia a partire dal *process* in figura 3.4. Si definiscono, in particolar modo, alcune variabili utilizzate nel processo, ovvero:

- I semi *seed1, seed2, seed3* e *seed4* utilizzati per generare numeri casuali (inizializzati rispettivamente a 55 e 5).
- Le variabili reali *r1* e *r2* che contengono i numeri casuali generati.
- I valori *max* e *min* per la generazione di numeri casuali.
- Le variabili intere *ia* e *ib* che rappresentano i valori interi generati.
- Le variabili *vA* e *vB* che contengono i numeri interi convertiti in std_logic_vector (8 bit).
- Il file di testo *ft* per salvare i risultati.
- La linea di testo *l* utilizzata per scrivere i dati nel file.

Inoltre, tramite il comando *file_open,* si apre il file "valori_integer.txt" in modalità di scrittura.

```
51    process
52        variable seed1, seed2: integer := 55;
53        variable seed3, seed4: integer := 5;
54        variable r1, r2 : real;
55        constant max: integer := 255;
56        constant min: integer := 0;
57        variable ia, ib: integer;
58        variable vA, vB: std_logic_vector(7 downto 0);
59        FILE ft: text;
60        variable l: line;
61    begin
62
63        file_open(ft,
64        "C:/Users/Utente/Desktop/INGEGNERIA/Tesi/Valori_casuali/valori_integer.txt",
65        write_mode);
```

Figura 3.4. Definizione delle variabili utilizzate nel testbench.

```
67        for i in 0 to 20000 loop
68
69            uniform(seed1, seed2, r1);
70            ia := integer(round(r1 * real(max-min) + real(min) - 0.5));
71            vA := conv_std_logic_vector(ia, 8);
72            uniform(seed3, seed4, r2);
73            ib := integer(round(r2 * real(max-min) + real(min) - 0.5));
74            vB := conv_std_logic_vector(ib, 8);
75
76            A <= vA;
77            B <= vB;
78            wait for 20 ns;
```

Figura 3.5. Generazione dei numeri e assegnazione degli ingressi dei moltiplicatori.

A questo punto, tramite un ciclo for, ripetuto per 20000 volte, si genera, utilizzando i semi, un numero casuale (tra 0 e 1) per ogni variabile. Tali numeri sono poi convertiti in un valore intero tra min (0) e max (255). Dopo aver convertito i valori interi in vettori, si assegnano i due risultati ai due ingressi A e B dei moltiplicatori (vedasi figura 3.4). Infine, si introduce un ritardo di 20 nanosecondi per consentire la simulazione nel tempo.

Una volta implementato ciò, si scrivono, utilizzando i comandi *write* e *writeline*, i valori di A, B e i due prodotti, ottenuti con il moltiplicatore esatto e approssimato, all'interno del file "valori_integer.txt", dopo averli convertiti in numeri interi tramite il comando *conv_integer.*

Dopo aver scritto tutti i dati, si chiude il file di testo e si mantiene attivo il processo (attesa indefinita) tramite il comando *wait*.

```
79              write(l, string'("Valore di A: "));
80              write(l, integer'(ia));
81              writeline(ft, l);
82              write(l, string'("Valore di B: "));
83              write(l, integer'(ib));
84              writeline(ft, l);
85              write(l, string'("Risultato esatto: "));
86
87              write(l, conv_integer(Ris_esatto));
88              writeline(ft, l);
89              write(l, string'("Risultato approssimato: "));
90              write(l, conv_integer(Ris_approssimato));
91              writeline(ft, l);
92          end loop;
93          file_close(ft);
94          wait;
95      end process;
96  end tbbrand_integer;
```

Figura 3.6. Scrittua dei prodotti all'interno del file.

Una volta avviata la simulazione, si ottengono i valori generati casualmente, scritti correttamente all'interno del file.

Consideriamo alcuni dei numeri generati, riportati in figura 3.7:

```
Valore di A: 8
Valore di B: 47
Risultato esatto: 376
Risultato approssimato: 572
Valore di A: 228
Valore di B: 206
Risultato esatto: 46968
Risultato approssimato: 47164
Valore di A: 141
Valore di B: 82
Risultato esatto: 11562
Risultato approssimato: 11630
Valore di A: 167
Valore di B: 107
Risultato esatto: 17869
Risultato approssimato: 17925
Valore di A: 58
Valore di B: 5
Risultato esatto: 290
Risultato approssimato: 446
```

Fiugra 3.7. Valori generati all'interno del file "valori_integer.txt".

Si noti come, per prodotti con un maggior numero di cifre, la differenza tra l'errore approssimato e l'errore esatto risulta minore rispetto ai prodotti con un minor numero di cifre.

3.2. Prodotto di immagini

Si ripete lo stesso esperimento applicando il moltiplicatore a virgola fissa approssimato ad un caso concreto: il prodotto tra due immagini.

Consideriamo due coppie di immagini da dover moltiplicare

Tali immagini sono convertite, tramite il software “gimp”, in file ASCII in formato .pgm.



Figura 3.8. Prima coppia di immagini da moltiplicare





Figura 3.9. Seconda coppia di immagini da moltiplicare

Si analizza l'intestazione delle tre immagini (comune a tutte e tre).

```
P2
# Created by GIMP version 2.10.20 PNM plug-in
288 180
255
```

Figura 3.10. Prima coppia di immagini da moltiplicare

La prima riga indica che il file è in formato PGM ASCII (Portable Gray Map), dove i valori dei pixel sono rappresentati in forma testuale. La seconda è una riga di commento (le righe di commento in PGM iniziano con il simbolo #) e indica che l'immagine è stata creata o modificata utilizzando il software di grafica GIMP, specificando anche la versione del programma (2.10.20) e il plug-in utilizzato per salvare l'immagine nel formato PNM (Portable Anymap Format, che include PGM, PPM e PBM). La terza riga indica le dimensioni dell'immagine in pixel. Il primo numero, 288, è la larghezza (numero di colonne), mentre il secondo, 180, è l'altezza (numero di righe). La quarta riga, infine, rappresenta il massimo livello di grigio che un pixel può avere. Essendo un'immagine in scala di grigi, i pixel possono avere valori da 0 (nero) a 255 (bianco). Questo valore determina la profondità di colore (8 bit, in questo caso).

Per il prodotto delle due coppie di immagini si utilizza un testbench, come nel caso precedente.

Cio che cambia in questo secondo esperimento è il process all'interno dell'architettura.

```
50      process
51          variable A_value, B_value: integer;
52          variable result_value1, result_value2: integer;
53          FILE image1_file, image2_file, result_file1, result_file2: text;
54          variable line1, line2: line;
55          variable header line: line;
```

Figura 3.11. Definizione delle variabili utilizzate nel process del testbench per il prodotto di immagini.

Vengono definite le seguenti variabili:

- "A_value" e "B_value", che immagazzinano temporaneamente i valori letti dai file delle immagini.
- "result_value1" e "result_value2", le quali contengono i valori interi dei risultati esatti e approssimati.
- "image1_file" e "image2_file", ovvero i file delle immagini da leggere.
- "result_file1" e "result_file2", i file dove, invece, vengono scritti i risultati delle moltiplicazioni (esatti e approssimati).
- "line1", "line2" e header line", variabili di tipo *line* utilizzate per gestire le righe durante la lettura/scrittura.

```
56      begin
57          file_open(image1_file,
58          "C:/Users/Utente/Desktop/INGEGNERIA/Tesi/Nuove_immagini/th_1.pgm",
59          read_mode);
60          file_open(image2_file,
61          "C:/Users/Utente/Desktop/INGEGNERIA/Tesi/Nuove_immagini/th_2.pgm",
62          read_mode);
63          file_open(result_file1,
64          "C:/Users/Utente/Desktop/INGEGNERIA/Tesi/Nuove_immagini/risultato_esatto_1x2.pgm",
65          write_mode);
66          file_open(result_file2,
67          "C:/Users/Utente/Desktop/INGEGNERIA/Tesi/Nuove_immagini/risultato_approssimato_1x2.pgm",
68          write_mode);
69
```

Figura 3.12. Apertura dei file utilizzate nel testbench per il prodotto di immagini.

I file "th_1.pgm" e "th_2.pgm" vengono aperti in modalità di lettura, mentre i file contenenti i risultati, cioè "risultato_esatto_1x2" e "risultato_approssimato_1x2", vengono aperti in modalità di scrittura.

Ovviamente, per eseguire il prodotto tra la seconda coppia di immagini, si apre il file “th_3.pgm” al posto del file “th_2.pgm”.

```
71          readline(image1_file, header_line);
72          readline(image1_file, header_line);
73          readline(image1_file, header_line);
74          readline(image1_file, header_line);
75
76          readline(image2_file, header_line);
77          readline(image2_file, header_line);
78          readline(image2_file, header_line);
79          readline(image2_file, header_line);
```

Figura 3.13. Lettura delle intestazioni dei file

Vengono letti 4 header (le prime 4 righe) di ciascun file di immagine per saltare l'intestazione, come da specifica PGM.

Si leggono, successivamente, i valori di ogni pixel (ogni riga contiene un pixel).

```
82          while not endfile(image1_file) and not endfile(image2_file) loop
83              readline(image1_file, line1);
84              readline(image2_file, line2);
85              read(line1, A_value);
86              read(line2, B_value);
87
88              A <= conv_std_logic_vector(A_value, 8);
89              B <= conv_std_logic_vector(B_value, 8);
90
91              wait for 0.001 ns;
92
93              result_value1 := conv_integer(unsigned(Ris_esatto));
94              result_value2 :=conv_integer(unsigned(Ris_approssimato));
95              write(line1, result_value1);
96              writeline(result_file1, line1);
97              write(line1, result_value2);
98              writeline(result_file2, line1);
99          end loop;
```

Figura 3.14. Lettura dei pixel delle immagini e scrittura su file del loro prodotto approssimato ed esatto.

Un ciclo while, fino alla fine di entrambi i file, legge una riga da ciascuno e la memorizza in “line1” e “line2”. Successivamente, tramite il comando *read*, viene

convertito il contenuto della riga letta in valori interi (A_value, B_value), i quali vengono passati ai segnali A e B per essere moltiplicati dai due componenti (esatto e approssimato).

Tramite la linea di codice *wait for 0.001 ns* si aspetta una piccola quantità di tempo per la simulazione.

Infine, i risultati vengono convertiti in numeri interi e scritti nei file di output (uno per il risultato esatto, uno per quello approssimato).

```
101             file_close(image1_file);
102             file_close(image2_file);
103             file_close(result_file1);
104             file_close(result_file2);
105             wait;
106         end process;
107     end behavior;
```

Figura 3.15. Chiusura dei file contenenti i pixel delle due immagini singole e dell'immagine risultato.

Una volta terminato il ciclo, i file vengono chiusi.

Ai due file "Risultato_esatto_1x2.pgm" e "Risultato_approssimato_1x2,pgm" bisogna aggiungere manualmente l'intestazione, per poter visualizzare le immagini risultanti. L'unica linea dell'intestazione diversa rispetto a quella delle due immagini da moltiplicare riguarda il massimo livello di grigio che un pixel può avere. Infatti, avendo utilizzato, moltiplicatori a 16 bit, bisogna impostare quest'ultimo a 65535 ($2^{16} - 1$).

Nelle figure 3.16 e 3.17 sono mostrati, rispettivamente, i prodotti esatti e approssimati tra la prima e la seconda immagine, e tra la prima e la terza immagine.



Figura 3.16. Prodotto esatto e approssimato tra la prima e seconda immagine.

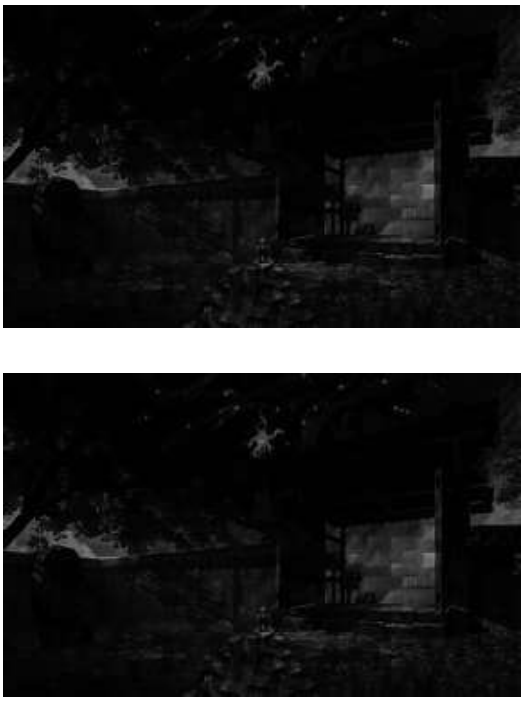


Figura 3.17. Prodotto esatto e approssimato tra la prima e terza immagine

In entrambi i prodotti, a livello visivo, le immagini risultanti dalla moltiplicazione dei pixel, ottenute con un moltiplicatore esatto ed uno approssimato, appaiono visivamente indistinguibili, poichè l'errore introdotto si distribuisce in modo diffuso e irrilevante rispetto alla complessità della struttura visiva complessiva. Tale tolleranza risulta di grande beneficio nelle applicazioni grafiche, dove la fedeltà percettiva può essere mantenuta anche in presenza di approssimazioni computazionali, consentendo di raggiungere un equilibrio tra precisione aritmetica e efficienza elaborativa.

3.3. Calcolo dell'errore medio ed errore medio normalizzato

L'utilizzo del moltiplicatore a virgola fissa approssimato per i prodotti trattati nei paragrafi precedenti causa alcuni errori dovuti alla differenza tra i valori approssimati e i valori esatti, ottenuti con il moltiplicatorie esatto.

Per ognuno degli esperimenti si calcola, in particolar modo, l'errore medio e l'errore medio normalizzato, definiti, rispettivamente, dalle formule:

$$ME\ (Mean\ Error) = \frac{1}{N}\sum_{i=1}^{N}\left(P_{approssimato}(i) - P_{esatto}(i)\right)$$

$$NME\ (Normalized\ Mean\ Error) = \frac{\frac{1}{N}\sum_{i=1}^{N}\left|P_{approssimato}(i) - P_{esatto}(i)\right|}{\frac{1}{N}\sum_{i=1}^{N}\left|P_{esatto}(i)\right|}$$

Nello specifico, N rappresenta il numero totale di moltiplicazioni, $P_{esatto}(i)$ il risultato esatto della i-esima moltiplicazione e $P_{approssimato}(i)$ il risultato approssimato della i-esima moltiplicazione.

Si calcolano tali errori tramite due programmi scritti attraverso il linguaggio di programmazione c [13], i quali, dopo aver letto i valori esatti ed approssimati dai

file di testo, applicano le formule definite in precedenza per il calcolo, il quale viene stampato su console.

Si analizza il programma in c [13], scritto attraverso il software “Visual Studio” [14], relativo al calcolo dell’errore medio ed errore medio normalizzato per il prodotto tra i numeri generati casualmente dal tesbench relativo al paragrafo 3.1.

```
15      char fileName[] = "valori_integer.txt";
16      file = fopen(fileName, "r");
17      if (file == NULL) {
18          printf("Errore nell'apertura del file.\n");
19          return 1;
20      }
```

Figura 3.18. Apertura del file “valori_integer.txt”.

Il file "valori_integer.txt" viene aperto in modalità di lettura. Se il file non esiste o non può essere aperto, viene stampato un messaggio di errore e il programma termina.

```
22      while (fscanf(file,
23          "Valore di A: %d\nValore di B: %d\nRisultato esatto: %lf\nRisultato approssimato: %lf\n",
24          &a, &b, &esatto, &approssimato) != EOF) {
25          double differenza = approssimato - esatto;
26          double differenza_nme;
27          if (esatto != 0) {
28              differenza_nme = fabs(esatto - approssimato) / fabs(esatto);
29          }
30          else differenza_nme = 0;
31          erroreMedio += differenza;
32          nme += differenza_nme;
33          count++;
34      }
```

Figura 3.19. Lettura dei valori dal file “valori_integer.txt” e calcolo degli errori.

Il ciclo while legge le righe del file usando la funzione *fscanf*. Calcola la differenza tra il valore approssimato e quello esatto e la differenza normalizzata (NME) solo se il risultato esatto non è zero, usando il rapporto tra il valore assoluto della differenza e il valore esatto. Gli errori vengono accumulati nelle variabili “erroreMedio” e “nme”. Viene incrementato il contatore count per tenere traccia del numero di dati processati.

Come si evince dalla figura 3.20, se sono stati letti dei dati (cioè count > 0), si calcolano l'errore medio e l'NME dividendo il valore accumulato per il numero di dati (count). I risultati vengono stampati con una precisione di 6 cifre decimali. Se non sono stati letti dati, viene stampato un messaggio che informa l'utente.

Una volta finita la lettura e l'elaborazione, il file viene chiuso con la funzione *fclose* (file) e il programma termina correttamente restituendo 0.

```
36      if (count > 0) {
37          erroreMedio /= count;
38          nme /= count;
39
40          printf("Errore Medio: %.6lf\n", erroreMedio);
41          printf("Errore Medio normalizzato (NME): %.6lf\n", nme);
42      }
43      else {
44          printf("Nessun dato trovato nel file.\n");
45      }
46
47
48      fclose(file);
49
50      return 0;
51  }
52
```

Figure 3.20. Stampa degli errori e chiusura del file.

Si analizza, adesso, il programma in c per il calcolo dell'errore medio e dell'errore medio normalizzato nel caso del prodotto delle immagini. La struttura del programma è analoga alla precedente. Ciò che cambia sono i due file da aprire tramite la funzione *fopen,* oltre al fatto che, in questo caso, i valori sono letti da due file differenti.

```
8       char esattoFileName[] = "risultato_esatto_1x2.pgm";
9       char approssimatoFileName[] = "risultato_approssimato_1x2.pgm";
10
11      fileEsatto = fopen(esattoFileName, "r");
12      fileApprossimato = fopen(approssimatoFileName, "r");
13
14      if (fileEsatto == NULL || fileApprossimato == NULL) {
15          printf("Errore nell'apertura dei file.\n");
16          return 1;
17      }
```

Figura 3.21. Apertura dei due file .pgm contenenti il prodotto tra le immagini "th_1.pgm" e "th_2.pgm".

Se uno dei due file non può essere aperto (ad esempio se non esiste), viene stampato un messaggio di errore e il programma termina.

Ovviamente, volendo calcolare l'errore medio ed errore medio normalizzato relativo al prodotto esatto ed approssimato tra le immagini "th_1.pgm" e "th_3.pgm", bisogna cambiare i nomi dei file "risultato_esatto_1x2.pgm" e "risultato_approssimato_1x2.pgm".

L'implementazione successiva del programma è analoga a quanto visto in precedenza.

Eseguendo i due file si ottengono i valori riportati in tabella 3.1. Essi risultano accettabili, considerando l'elevato numero di cifre dei prodotti ottenuti con i due moltiplicatori. L'errore medio normalizzato dell'ultimo prodotto è, invece, piuttosto alto. Ma la causa è dovuta ad un elevato numero di zeri presenti nel file "risultato_esatto_1x3.pgm".

Tabella 3.1. Errore medio ed errrore medio normalizzato tra i valori esatti ed approssimati ottenuti dai due moltiplicatori

	Errore medio	Errore medio normalizzato
Prodotto tra i numeri generati casualmente.	120.080000	0.091411
Prodotto tra la prima e la seconda immagine.	137.617515	0.176686
Prodotto tra la prima e la terza immagine.	146.995216	0.773458

CONCLUSIONI

Nella tesi è stata affrontata l'implementazione di un moltiplicatore a virgola fissa approssimato, con l'obiettivo di valutare i compromessi tra precisione e risparmio in termini di area, tempo e consumo energetico.

Attraverso un'analisi dettagliata è emerso che i moltiplicatori esatti, sebbene garantiscano una precisione maggiore, presentano limitazioni in contesti dove l'efficienza e la velocità sono cruciali, soprattutto per applicazioni con risorse hardware limitate, come i dispositivi portatili o IoT.

L'introduzione del moltiplicatore approssimato ha mostrato vantaggi significativi, in particolare per quanto riguarda la riduzione dell'area occupata e del consumo energetico, grazie all'uso di adders e compressori semplificati. Questa semplificazione ha introdotto errori trascurabili in molte applicazioni tolleranti, come l'elaborazione di immagini e segnali. I test effettuati sia su numeri generati casualmente che su immagini hanno evidenziato una minima differenza tra i risultati esatti e quelli approssimati, dimostrando che il moltiplicatore approssimato rappresenta una soluzione efficiente per contesti in cui la tolleranza agli errori è accettabile.

Infine, l'applicazione delle tecniche di correzione degli errori, come l'integrazione del modulo ECM, ha ulteriormente migliorato la precisione del sistema, permettendo di bilanciare efficienza e accuratezza. Le simulazioni hanno confermato che, anche in presenza di approssimazioni, i risultati ottenuti risultano sufficientemente accurati per applicazioni grafiche o multimediali, dove l'impatto visivo dell'approssimazione è irrilevante.

In conclusione, si è dimostrato che l'uso di moltiplicatori approssimati può portare a notevoli benefici in termini di risparmio di risorse, senza compromettere significativamente la qualità dei risultati, rendendoli ideali per applicazioni embedded e multimediali.

INDICE DELLE FIGURE

Figura 1.1. Matrice dei prodotti parziali [1], [4]. 4
Figura 1.2 Circuito logico di un half adder. 5
Figura 1.3. Circuito logico di un full adder 5
Figura 1.4. Circuito logico relativo ad un compressore 4-2. 6
Figura 1.5. Logica dell'implementazione del moltiplicatore esatto. 7
Figura 1.6. Implementazione VHDL dell'half adder. 8
Figura 1.7. Implementazione VHDL del full adder 8
Figura 1.8. Implementazione VHDL del compressore 4-2. 9
Figura 1.9. Definizione delle librerie e dell'entity del moltiplicatore esatto ad 8 bit. 10
Figura 1.10. Definizione delle componenti utilizzate. 10
Figura 1.11. Dichiarazioni di tipi e segnali. 11
Figura 1.12. Generazione dei prodotti parziali. 11
Figura 1.13. Primo stadio di compressione dei prodotti parziali per un moltiplicatore esatto. 12
Figura 1.14. Secondo stadio di compressione e assegnazione del vettore "Product" per un moltiplicatore esatto 13
Figura 1.15. Simulazione di un moltiplicatore esatto a 8 bit. 13
Figura 1.16. Layout del dispositivo del moltiplicatore esatto 14
Figura 1.17. Analisi della potenza del moltiplicatore esatto 14
Figura 1.18. Schema logico del moltiplicatore esatto. 15
Figura 1.19. Utilizzo delle risorse del moltiplicatore esatto. 15
Figura 2.1. Circuito logico di un half adder approssimato. 18
Figura 2.2. Circuito logico di un full adder approssimato. 19
Figura 2.3. Circuito logico di un NAC. 20
Figura 2.4. Circuito logico del modulo di correzione degli errori. 23
Figura 2.5. Definizione delle librerie adoperate e dell'entity del moltiplicatore a virgola fissa approssimato. 24
Figura 2.6. Implementazione VHDL dell'half adder approssimato. 25

Figura 2.7. Implementazione VHDL del full adder approssimato. 25
Figura 2.8. Implementazione VHDL del compressore NAC.....26
Figura 2.9. Implementazione VHDL del modulo di correzione degli errori. 26
Figura 2.10. Dichiarazione dell'uso di componenti approssimate e del modulo di correzione degli errori.....27
Figura 2.11. Logica della compressione della matrice di bit dei prodotti parziali del moltiplicatore approssimato.....28
Figura 2.12. Primo stadio di compressione della matrice di prodotti parziali per il moltiplicatore approssimato.....29
Figura 2.13. Secondo stadio di compressione della matrice di prodotti parziali per il moltiplicatore approssimato. 29
Figura 2.14. Assegnazione finale dell'array "Product" per un moltiplicatore approssimato. 30
Figura 2.15. Simula zione del prodotto di alcuni array a 8 bit tramite un moltiplicatore approssimato.....31
Figura 2.16. Utilizzo delle risorse del moltiplicatore approssimato. 31
Figura 2.17. Layout del dispositivo del moltiplicatore aprprossimato. 32
Figura 2.18. Analisi della potenza del moltiplicatore approssimato.....32
Figura 2.19. Schema logico del moltiplicatore approssimato.....33
Figura 3.1. Definizione delle librerie e dell'entity del testbench per il prodotto di numeri generati casualmente. 35
Figura 3.2. Definizione delle componenti utilizzate nel testbench.....35
Figura 3.3. Instanza dei due moltiplicatori utilizzati nel testbench. 36
Figura 3.4. Definizione delle variabili utilizzate nel testbench. 37
Figura 3.5. Generazione dei numeri e assegnazione degli ingressi dei moltiplicatori. 37
Figura 3.6. Scrittua dei prodotti all'interno del file. 38
Fiugra 3.7. Valori generati all'interno del file "valori_integer.txt".....39
Figura 3.8. Prima coppia di immagini da moltiplicare 40
Figura 3.9. Seconda coppia di immagini da moltiplicare 41
Figura 3.10. Prima coppia di immagini da moltiplicare 41

Figura 3.11. Definizione delle variabili utilizzate nel process del testbench per il prodotto di immagini. 42
Figura 3.12. Apertura dei file utilizzate nel testbench per il prodotto di immagini. 42
Figura 3.13. Lettura delle intestazioni dei file 43
Figura 3.14. Lettura dei pixel delle immagini e scrittura su file del loro prodotto approssimato ed esatto. 43
Figura 3.15. Chiusura dei file contenenti i pixel delle due immagini singole e dell'immagine risultato. 44
Figura 3.16. Prodotto esatto e approssimato tra la prima e seconda immagine.....45
Figura 3.17. Prodotto esatto e approssimato tra la prima e terza immagine.....45
Figura 3.18. Apertura del file "valori_integer.txt". 47
Figura 3.19. Lettura dei valori dal file "valori_integer.txt" e calcolo degli errori.47
Figure 3.20. Stampa degli errori e chiusura del file.....48
Figura 3.21. Apertura dei due file .pgm contenenti il prodotto tra le immagini "th_1.pgm" e "th_2.pgm". 48

INDICE DELLE TABELLE

Tabella 1.1. Tabella della verità relativa ad un half adder. 5
Tabella 1.2. Tabella della verità relativa ad un full adder. 6
Tabella 2.1. Tabella della verità di un half adder approssimato e confronto con l'half adder esatto in termini di errore. 19
Tabella 2.2. Tabella della verità di un full adder approssimato e confronto con il full adder esatto in termini di errore. 20
Tabella 2.3. Tabella della verità del compressore NAC e confronto con il compressore 4-2 esatto in termini di errore. 21
Tabella 2.4. Confronto prestazionale tra il moltiplicatore esatto e approssimato.. 33
Tabella 3.1. Errore medio ed errrore medio normalizzato tra i valori esatti ed approssimati ottenuti dai due moltiplicatori 49

BIBLIOGRAFIA

[1]. Singh, K., Puligilla, S. V., Ramesh, H., Lnu, A., & El-Razouk, H. (2024). *Efficient designs for the 8-bit approximate fixed point multiplication.* 14th Annual Computing and Communication Workshop and Conference (CCWC), https://doi.org/10.1109/CCWC60891.2024.10427674.

[2]. IEEE Standards Association. (2019). *IEEE Standard VHDL Language Reference Manual* (IEEE Std 1076-2019). Institute of Electrical and Electronics Engineers. https://ieeexplore.ieee.org/document/8938196.

[3]. Xilinx Inc. (2022). *Vivado Design Suite* [Software]. https://www.xilinx.com/products/design-tools/vivado.html.

[4]. B. Parhami. (2010). *Computer Arithmetic: Algorithms and Hardware Designs*. 2nd edition, Oxford University Press, New York.

[5]. H. Pei, X. Yi, H. Zhou, and Y. He. (2021). *Design of ultra-low power consumption approximate 4-2 compressors based on the compensation characteristic*, IEEE Transactions on Circuits and Systems II: Express Briefs, vol. 68, no. 1.

[6]. A. Momeni, J. Han, P. Montuschi, and F. Lombardi. (2015). *Design and analysis of approximate compressors for multiplication*, IEEE Transactions on Computers, vol. 64, no. 4.

[7]. Z. Yang, J. Han, and F. Lombardi. (2015). *Approximate compressors for errorresilient multiplier design*, in 2015 IEEE International Symposium on Defect and Fault Tolerance in VLSI and Nanotechnology Systems (DFTS).

[8]. T. Kong and S. Li. (2021). *Design and analysis of approximate 4-2 compressors for high-accuracy multipliers,* IEEE Transactions on Very Large Scale Integration (VLSI) Systems, vol. 29, no. 10.

[9]. G. Vadhyan, A. R. R. S, and M. R. (2022). *Design of low power & high speed approximate compressor based multiplier*, in 2022 2nd Asian Conference on Innovation in Technology (ASIANCON).

[10]. M. Ha and S. Lee. (2018). *Multipliers with approximate 4-2 compressors and error recovery modules*, IEEE Embedded Systems Letters, vol. 10, no. 1.

[11]. J. Xu, Y. Li, X. Hong, J. Qiu, Y. Zuo, W. Li, H. Guo, and J. Wu. (2021). *Multiplier-free parallel fixed-point adaptive equalizer for real-time digital coherent communication*, IEEE Communications Letters, vol. 25, no. 7.

[12]. Y. Guo, H. Sun, L. Guo, and S. Kimura. (2018). *Low-cost approximate multiplier design using probability-driven inexact compressors*, in 2018 IEEE Asia Pacific Conference on Circuits and Systems (APCCAS).

[13]. HTML.it. (n.d.). *Guida linguaggio C*. https://www.html.it/guide/guida-c/.

[14]. Microsoft. (n.d.). *Visual Studio*. https://visualstudio.microsoft.com/it/vs/.